The Concise TypeScript Book

Simone Poggiali

The Concise TypeScript Book

The Concise TypeScript Book은 TypeScript의 기능을 포괄적이면서도 간결하게 설명합니다. 강력한 타입 시스템부터 고급 기능까지 최신 버전의 언어가 제공하는 모든 측면을 명확하게 다룹니다.

초보자든 숙련된 개발자든, 이 책은 TypeScript에 대한 이해와 숙련도를 높이는 데 매우 유용한 자료입니다.

이 책은 완전히 무료이며 오픈 소스입니다.

저는 수준 높은 기술 교육을 누구나 접할 수 있어야 한다고 믿습니다. 이러한 이유로 책을 무료로 제공하며 개선 사항과 새로운 예제를 반영해 정기적으로 업데이트합니다.

The Concise TypeScript Book Plus Edition을 만나 보세요.



오픈 소스 에디션을 넘어 더 깊이 학습하고 싶은 독자를 위해 **The Concise TypeScript Book Plus Edition: React and Real-World Patterns for TypeScript 7**에는 실무 적용에 초점을 맞춘 추가 독점 콘텐츠가 포함되어 있습니다.

Plus Edition에는 다음 내용이 포함됩니다.

- **TypeScript 7에 맞춰 업데이트** — 최신 TypeScript 7 기능과 언어 개선 사항을 다룹니다.
- **React와 함께 사용하는 TypeScript** — 컴포넌트, props, 훅, 이벤트, children, ref 및 일반적인 React 패턴의 타입을 지정하는 방법을 실용적으로 안내합니다.

- **실무 프로젝트를 위한 TypeScript 레시피** — 개발자가 TypeScript 애플리케이션을 구축하고 유지 관리할 때 마주치는 실질적인 문제를 다루는 핵심 예제를 제공합니다.

Plus Edition을 구매하면 무료 오픈 소스 도서의 지속적인 개발과 유지 관리도 직접 후원하게 됩니다.

Plus Edition은 전 세계 Amazon에서 영어와 이탈리아어로 제공됩니다. Plus Edition을 살펴보고 Amazon에서 구매하기.

프로젝트 후원하기

무료 도서가 버그를 수정하거나 어려운 개념을 이해하거나 경력을 발전시키는 데 도움이 되었다면, 권장 후원 금액인 **$5**를 참고해 원하는 금액을 후원하거나 커피 한 잔을 사 주는 방식으로 제 작업을 지원해 주세요.

여러분의 후원은 콘텐츠를 최신 상태로 유지하고 새로운 예제, 더 명확한 설명, 추가적인 실용 지침으로 확장하는 데 도움이 됩니다.





번역

이 책은 다음을 포함한 여러 언어로 번역되었습니다.

영어

불가리아어

독일어

프랑스어

인도네시아어

이탈리아어

일본어

한국어

폴란드어

포르투갈어(브라질)

스웨덴어

터키어

Vietnamese

중국어

스페인어

태국어

러시아어

아랍어 체코어

다운로드 및 웹사이트

EPUB 버전도 다운로드할 수 있습니다.

https://github.com/gibbok/typescript-book/tree/main/downloads

온라인 버전은 다음에서 이용할 수 있습니다.

https://gibbok.github.io/typescript-book

목차

- The Concise TypeScript Book
  - 프로젝트 후원하기
  - 번역
  - 다운로드 및 웹사이트
  - 목차
  - 소개
  - 저자 소개
  - TypeScript 소개
    - TypeScript란?
    - 왜 TypeScript인가?
    - TypeScript와 JavaScript
    - TypeScript 코드 생성
    - 현대적인 JavaScript를 지금 사용하기(다운레벨링)
  - TypeScript 시작하기
    - 설치
    - 구성
    - TypeScript 구성 파일
      - target
      - lib
      - strict
      - module

        - moduleResolution
        - esModuleInterop
        - jsx
        - skipLibCheck
        - files
        - include
        - exclude
    - importHelpers
    - TypeScript 마이그레이션 조언
- 타입 시스템 살펴보기
    - TypeScript 언어 서비스
    - 구조적 타이핑
    - TypeScript의 기본 비교 규칙
    - 집합으로서의 타입
    - 타입 지정하기: 타입 선언과 타입 단언
        - 타입 선언
        - 타입 단언
        - 앰비언트 선언
    - 프로퍼티 검사와 초과 프로퍼티 검사
    - 약한 타입
    - 엄격한 객체 리터럴 검사(Freshness)
    - 타입 추론
    - 더 고급 추론
    - 타입 넓히기
    - Const
        - 타입 매개변수의 Const 수정자
        - Const 단언
    - 명시적 타입 어노테이션
    - 타입 좁히기

        - 조건
        - 예외 발생 또는 반환
        - 판별 유니온
        - 사용자 정의 타입 가드
        - Switch-true 좁히기
    - 원시 타입
        - string
        - boolean
        - number
        - bigint
        - Symbol
        - null과 undefined
        - Array
        - any
    - 타입 어노테이션
    - 선택적 프로퍼티
    - 읽기 전용 프로퍼티
    - 인덱스 시그니처
    - 타입 확장
    - 리터럴 타입
    - 리터럴 추론
    - strictNullChecks
    - 열거형
        - 숫자 열거형
        - 문자열 열거형
        - 상수 열거형
        - 역방향 매핑
        - 앰비언트 열거형
        - 계산된 멤버와 상수 멤버

- 좁히기
  - typeof 타입 가드
  - 참 같은 값에 의한 좁히기
  - 동등성 좁히기
  - In 연산자 좁히기
  - instanceof 좁히기
- 할당
- 제어 흐름 분석
- 타입 술어
- 판별 유니온
- never 타입에 대하여
- 완전성 검사
- 객체 타입
- 튜플 타입(익명)
- 명명된 튜플 타입(레이블 지정)
- 고정 길이 튜플
- 유니온 타입
- 인터섹션 타입
- 타입 인덱싱
- 값에서 타입 얻기
- 함수 반환값에서 타입 얻기
- 모듈에서 타입 얻기
- 매핑된 타입
- 매핑된 타입 수정자
- 조건부 타입
- 분배 조건부 타입
- 조건부 타입의 infer 타입 추론
- 미리 정의된 조건부 타입
- 템플릿 유니온 타입

- Any 타입
- Unknown 타입
- Void 타입
- Never 타입
- 인터페이스와 타입
  - 공통 구문
  - 기본 타입
  - 객체와 인터페이스
  - 유니온 타입과 인터섹션 타입
- 내장 원시 타입
- 일반적인 내장 JS 객체
- 오버로드
- 병합과 확장
- 타입과 인터페이스의 차이점
- 클래스
  - 클래스 공통 구문
  - 생성자
  - Private 및 Protected 생성자
  - 접근 제한자
  - Get과 Set
  - 클래스의 자동 접근자
  - this
  - 매개변수 프로퍼티
  - 추상 클래스
  - 제네릭과 함께 사용하기
  - 데코레이터
    - 클래스 데코레이터
    - 프로퍼티 데코레이터
    - 메서드 데코레이터

        - Getter 및 Setter 데코레이터
        - 데코레이터 메타데이터
    - 상속
    - 정적 멤버
    - 프로퍼티 초기화
    - 메서드 오버로딩
- 제네릭
    - 제네릭 타입
    - 제네릭 클래스
    - 제네릭 제약 조건
    - 제네릭의 문맥적 타입 좁히기
- 소거된 구조적 타입
- 네임스페이스 지정
- 심볼
- 트리플 슬래시 지시문
- 타입 조작
    - 타입에서 타입 생성하기
    - 인덱스 접근 타입
    - 유틸리티 타입
        - Awaited<T>
        - Partial<T>
        - Required<T>
        - Readonly<T>
        - Record<K, T>
        - Pick<T, K>
        - Omit<T, K>
        - Exclude<T, U>
        - Extract<T, U>
        - NonNullable<T>

        - Parameters<T>
        - ConstructorParameters<T>
        - ReturnType<T>
        - InstanceType<T>
        - ThisParameterType<T>
        - OmitThisParameter<T>
        - ThisType<T>
        - Uppercase<T>
        - Lowercase<T>
        - Capitalize<T>
        - Uncapitalize<T>
        - NoInfer<T>
    - 기타
        - 오류 및 예외 처리
        - 믹스인 클래스
        - 비동기 언어 기능
        - 이터레이터와 제너레이터
        - TsDocs JSDoc 참조
        - @types
        - JSX
        - ES6 모듈
        - ES7 지수 연산자
        - for-await-of 문
        - new.target 메타 프로퍼티
        - 동적 import 표현식
        - “tsc -watch”
        - Non-null 단언 연산자
        - 기본값 선언
        - 옵셔널 체이닝

- 널 병합 연산자
- 템플릿 리터럴 타입
- 함수 오버로딩
- 재귀 타입
- 재귀 조건부 타입
- Node의 ECMAScript 모듈 지원
- 단언 함수
- 가변 튜플 타입
- 박싱 타입
- TypeScript의 공변성과 반공변성
  - 타입 매개변수의 선택적 변성 어노테이션
- 템플릿 문자열 패턴 인덱스 시그니처
- satisfies 연산자
- 타입 전용 가져오기와 내보내기
- using 선언과 명시적 리소스 관리
  - await using 선언
- 가져오기 속성
- 정규 표현식 구문 검사
- import defer

소개

The Concise TypeScript Book에 오신 것을 환영합니다! 이 가이드는 효과적인 TypeScript 개발에 필요한 필수 지식과 실무 기술을 제공합니다. 깔끔하고 견고한 코드를 작성하기 위한 핵심 개념과 기법을 알아보세요. 초보자든 숙련된 개발자든, 이 책은 프로젝트에서 TypeScript의 강력한 기능을 활용하는 데 도움이 되는 종합 안내서이자 편리한 참고 자료입니다.

이 책은 TypeScript 7.0을 다룹니다.

저자 소개

Simone Poggiali는 90년대부터 전문가 수준의 코드 작성에 열정을 쏟아 온 숙련된 Staff Engineer입니다. 그는 국제적인 경력을 쌓는 동안 스타트업부터 대규모 조직까지 다양한 고객을 위한 수많은 프로젝트에 기여했습니다. HelloFresh, Siemens, O2, Leroy Merlin, Snowplow와 같은 유명 기업들이 그의 전문성과 헌신 덕분에 많은 도움을 받았습니다.

다음 플랫폼에서 Simone Poggiali에게 연락할 수 있습니다.

- LinkedIn: https://www.linkedin.com/in/simone-poggiali
- GitHub: https://github.com/gibbok
- Bluesky: https://bsky.app/profile/gibbok.bsky.social
- 이메일: gibbok.coding@gmail.com

전체 기여자 목록: https://github.com/gibbok/typescript-book/graphs/contributors

TypeScript 소개

TypeScript란?

TypeScript는 JavaScript를 기반으로 구축된 강타입 프로그래밍 언어입니다. 원래 2012년에 Anders Hejlsberg가 설계했으며, 현재 Microsoft가 오픈 소스 프로젝트로 개발하고 유지 관리하고 있습니다.

TypeScript는 JavaScript로 컴파일되며 모든 JavaScript 런타임(예: 브라우저 또는 서버의 Node.js)에서 실행할 수 있습니다.

함수형, 제네릭, 명령형 및 객체 지향 프로그래밍과 같은 여러 프로그래밍 패러다임을 지원하며, 실행 전에 JavaScript로 변환되는 컴파일(트랜스파일) 언어입니다.

왜 TypeScript인가?

TypeScript는 프로그램이 실행되기 전에 일반적인 프로그래밍 실수를 방지하고 특정 종류의 런타임 오류를 피할 수 있도록 도와주는 강타입 언어입니다.

강타입 언어를 사용하면 개발자가 데이터 타입 정의에 다양한 프로그램 제약 조건과 동작을 지정할 수 있으므로 소프트웨어의 정확성을 검증하고 결함을 방지하기가 쉬워집니다. 이는 특히 대규모 애플리케이션에서 유용합니다.

TypeScript의 몇 가지 장점은 다음과 같습니다.

- 정적 타이핑, 선택적인 강타입 지정
- 타입 추론
- ES6 및 ES7 기능 사용
- 크로스 플랫폼 및 크로스 브라우저 호환성
- IntelliSense를 통한 도구 지원

TypeScript와 JavaScript

TypeScript는 .ts 또는 .tsx 파일로 작성하는 반면, JavaScript 파일은 .js 또는 .jsx로 작성합니다.

확장자가 .tsx 또는 .jsx인 파일에는 UI 개발을 위해 React에서 사용하는 JavaScript 구문 확장인 JSX가 포함될 수 있습니다.

TypeScript는 구문 측면에서 JavaScript(ECMAScript 2015)의 타입이 지정된 상위 집합입니다. 모든 JavaScript 코드는 유효한 TypeScript 코드이지만, 그 반대가 항상 성립하는 것은 아닙니다.

예를 들어, 다음과 같이 확장자가 .js인 JavaScript 파일의 함수를 살펴보겠습니다.

```
const sum = (a, b) => a + b;
```

파일 확장자를 .ts로 변경하면 이 함수를 TypeScript로 변환하여 사용할 수 있습니다. 그러나 같은 함수에 TypeScript 타입 어노테이션을 추가하면 컴파일하지 않고는 어떤 JavaScript 런타임에서도 실행할 수 없습니다. 다음 TypeScript 코드는 컴파일하지 않으면 구문 오류가 발생합니다.

```
const sum = (a: number, b: number): number => a + b;
```

TypeScript는 개발자가 타입 어노테이션을 통해 의도를 표현할 수 있도록 하여 잠재적인 런타임 오류를 컴파일 시점에 감지하도록 설계되었습니다. 또한 타입 추론 덕분에 명시적인 타입 어노테이션이 없어도 특정 문제를 포착할 수 있습니다. 예를 들어 다음 코드 조각에는 어떤 TypeScript 타입도 지정되어 있지 않습니다.

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

이 경우 TypeScript는 오류를 감지하고 다음과 같이 보고합니다.

```
Property 'y' does not exist on type '{ x: number; }'.
```

TypeScript의 타입 시스템은 JavaScript의 런타임 동작에 큰 영향을 받습니다. 예를 들어 JavaScript에서 문자열 연결이나 숫자 덧셈을 수행할

수 있는 덧셈 연산자(+)는 TypeScript에서도 같은 방식으로 모델링됩니다.

```
const result = '1' + 1; // Result is of type string
```

TypeScript 개발 팀은 JavaScript의 비정상적인 사용을 의도적으로 오류로 표시하기로 결정했습니다. 예를 들어 다음의 유효한 JavaScript 코드를 살펴보겠습니다.

```
const result = 1 + true; // In JavaScript, the result is equal to 2
```

하지만 TypeScript는 오류를 발생시킵니다.

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

이 오류는 TypeScript가 타입 호환성을 엄격하게 적용하기 때문에 발생하며, 이 경우 숫자와 불리언 사이의 잘못된 연산을 식별합니다.

TypeScript 코드 생성

TypeScript 컴파일러에는 타입 오류 검사와 JavaScript 컴파일이라는 두 가지 주요 역할이 있습니다. 이 두 프로세스는 서로 독립적입니다. 타입은 컴파일 중에 완전히 제거되므로 JavaScript 런타임에서 코드 실행에 영향을 주지 않습니다. TypeScript는 타입 오류가 있더라도 JavaScript를 출력할 수 있습니다. 다음은 타입 오류가 있는 TypeScript 코드의 예입니다.

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // Argument of type 'string' is not assignable
to parameter of type 'number'.
```

하지만 여전히 실행 가능한 JavaScript 출력을 생성할 수 있습니다.

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

런타임에는 TypeScript 타입을 검사할 수 없습니다. 예를 들면 다음과 같습니다.

```
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
        // 'Dog' only refers to a type, but is being used as a value here.
        // ...
    }
};
```

컴파일 후에는 타입이 제거되므로 JavaScript에서는 이 코드를 실행할 방법이 없습니다. 런타임에 타입을 식별하려면 다른 메커니즘을 사용해야 합니다. TypeScript는 여러 가지 방법을 제공하며, 일반적으로 사용하는 방법 중 하나는 “태그된 유니온”입니다. 예를 들면 다음과 같습니다.

```
interface Dog {
    kind: 'dog'; // Tagged union
    bark: () => void;
}
interface Cat {
    kind: 'cat'; // Tagged union
```

```
    meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);
```

"kind" 프로퍼티는 런타임에 JavaScript 객체를 구별하는 데 사용할 수 있는 값입니다.

런타임의 값이 타입 선언에서 선언된 타입과 다른 타입을 갖는 것도 가능합니다. 예를 들어 개발자가 API 타입을 잘못 해석하여 잘못된 어노테이션을 지정한 경우입니다.

TypeScript는 JavaScript의 상위 집합이므로 "class" 키워드를 런타임에 타입과 값으로 사용할 수 있습니다.

```
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
        public bark: () => void
```

```
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};

const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

JavaScript에서 "class"에는 "prototype" 프로퍼티가 있으며, "instanceof" 연산자를 사용하면 생성자의 prototype 프로퍼티가 객체의 프로토타입 체인 어디에든 존재하는지 검사할 수 있습니다.

모든 타입이 제거되므로 TypeScript는 런타임 성능에 아무런 영향을 주지 않습니다. 하지만 TypeScript는 빌드 시간에 어느 정도 오버헤드를 발생시킵니다.

현대적인 JavaScript를 지금 사용하기(다운레벨링)

TypeScript는 ECMAScript 3(1999) 이후 출시된 모든 JavaScript 버전으로 코드를 컴파일할 수 있습니다. 즉, TypeScript는 최신 JavaScript 기능을 사용한 코드를 이전 버전으로 트랜스파일할 수 있으며, 이 과정을 다운레벨링이라고 합니다. 이를 통해 이전 런타임 환경과의 호환성을 최대한 유지하면서 현대적인 JavaScript를 사용할 수 있습니다.

이전 버전의 JavaScript로 트랜스파일할 때 TypeScript가 네이티브 구현보다 성능 오버헤드를 일으킬 수 있는 코드를 생성할 수도 있다는 점에 유의해야 합니다.

TypeScript에서 사용할 수 있는 현대적인 JavaScript 기능은 다음과 같습니다.

- AMD 스타일의 “define” 콜백이나 CommonJS의 “require” 문 대신 ECMAScript 모듈 사용.
- 프로토타입 대신 클래스 사용.
- “var” 대신 “let” 또는 “const”를 사용한 변수 선언.
- 전통적인 “for” 루프 대신 “for-of” 루프 또는 “.forEach” 사용.
- 함수 표현식 대신 화살표 함수 사용.
- 구조 분해 할당.
- 축약 프로퍼티/메서드 이름과 계산된 프로퍼티 이름.
- 기본 함수 매개변수.

이러한 현대적인 JavaScript 기능을 활용하면 개발자는 TypeScript에서 더 표현력 있고 간결한 코드를 작성할 수 있습니다.

TypeScript 시작하기

설치

Visual Studio Code는 TypeScript 언어를 훌륭하게 지원하지만 TypeScript 컴파일러는 포함하지 않습니다. TypeScript 컴파일러를 설치하려면 npm 또는 yarn 같은 패키지 관리자를 사용할 수 있습니다.

```
npm install typescript --save-dev
```

또는

```
yarn add typescript --dev
```

모든 팀원이 같은 TypeScript 버전을 사용하도록 생성된 lockfile을 커밋해야 합니다.

TypeScript 컴파일러를 실행하려면 다음 명령을 사용할 수 있습니다.

```
npx tsc
```

또는

```
yarn tsc
```

빌드 프로세스를 더 예측 가능하게 만들 수 있으므로 TypeScript는 전역이 아닌 프로젝트별로 설치하는 것이 좋습니다. 하지만 일회성으로 사용할 때는 다음 명령을 실행할 수 있습니다.

```
npx tsc
```

또는 전역으로 설치할 수 있습니다.

```
npm install -g typescript
```

Microsoft Visual Studio를 사용하는 경우 MSBuild 프로젝트를 위해 NuGet에서 TypeScript를 패키지로 가져올 수 있습니다. NuGet Package Manager Console에서 다음 명령을 실행합니다.

```
Install-Package Microsoft.TypeScript.MSBuild
```

TypeScript를 설치하면 두 개의 실행 파일이 설치됩니다. TypeScript 컴파일러인 “tsc”와 TypeScript 독립 실행형 서버인 “tsserver”입니다. 이 독립 실행형 서버에는 편집기와 IDE가 지능형 코드 완성을 제공하는 데 사용할 수 있는 컴파일러와 언어 서비스가 포함되어 있습니다.

또한 Babel(플러그인을 통해 사용)이나 swc와 같이 TypeScript와 호환되는 여러 트랜스파일러가 있습니다. 이러한 트랜스파일러를 사용하면 TypeScript 코드를 다른 대상 언어나 버전으로 변환할 수 있습니다.

TypeScript 7.0은 컴파일러와 언어 서비스의 네이티브 구현으로 Go로 다시 작성되었습니다. 공유 메모리 멀티스레딩과 기타 최적화를 사용하여 전체 빌드와 편집기 기능을 더 빠르게 만들고 개발 중 피드백 시간을 줄입니다.

일부 TypeScript 7.0 성능 기능은 조정할 수 있습니다. 타입 검사는 `--checkers`를 사용하여 병렬 워커에서 실행할 수 있습니다. 워커 수가 많으면 대규모 프로젝트의 속도가 빨라질 수 있지만 메모리를 더 많이 사용합니다. 다시 구축된 `--watch` 모드는 크로스 플랫폼 파일 감시를 개선합니다. TypeScript 7.0에는 아직 컴파일러 API가 포함되어 있지 않으므로(2026년 7월 기준), 여전히 TypeScript 6.0 API가 필요한 도구는 `@typescript/typescript6` 또는 npm 별칭을 사용하여 TypeScript 7.0과 함께 실행할 수 있습니다.

구성

TypeScript는 tsc CLI 옵션을 사용하거나 프로젝트 루트에 tsconfig.json이라는 전용 구성 파일을 배치하여 구성할 수 있습니다.

권장 설정이 미리 채워진 tsconfig.json 파일을 생성하려면 다음 명령을 사용할 수 있습니다.

```
tsc --init
```

로컬에서 `tsc` 명령을 실행하면 TypeScript는 가장 가까운 tsconfig.json 파일에 지정된 구성을 사용하여 코드를 컴파일합니다.

다음은 기본 설정으로 실행되는 CLI 명령의 몇 가지 예입니다.

```
tsc main.ts // Compile a specific file (main.ts) to JavaScript
tsc src/*.ts // Compile any .ts files under the 'src' folder to JavaScript
tsc app.ts util.ts --outfile index.js // Compile two TypeScript files (app.ts and util.ts) into a single JavaScript file (index.js)
```

TypeScript 구성 파일

tsconfig.json 파일은 TypeScript 컴파일러(tsc)를 구성하는 데 사용됩니다. 일반적으로 `package.json` 파일과 함께 프로젝트 루트에 추가합니다.

참고:

- tsconfig.json은 JSON 형식이지만 주석을 허용합니다.
- 명령줄 옵션 대신 이 구성 파일을 사용하는 것이 좋습니다.

다음 링크에서 전체 문서와 해당 스키마를 확인할 수 있습니다.

https://www.typescriptlang.org/tsconfig

https://www.typescriptlang.org/tsconfig/

다음은 일반적이고 유용한 구성 목록입니다.

target

“target” 프로퍼티는 TypeScript 코드를 어떤 ECMAScript 버전으로 내보내거나 컴파일할지를 지정하는 데 사용됩니다. 최신 브라우저에는 ES6가 좋은 선택입니다. 참고: ES5 지원은 TypeScript 6.0에서 더 이상 사용되지 않는 기능으로 지정되었으며 TypeScript 7.0에서는 더 이상 지원되지 않습니다.

lib

“lib” 프로퍼티는 컴파일할 때 포함할 라이브러리 파일을 지정하는 데 사용됩니다. TypeScript는 “target” 프로퍼티에 지정된 기능의 API를 자동으로 포함하지만, 특정 요구 사항에 따라 특정 라이브러리를 생략하거나 선택할 수 있습니다. 예를 들어 서버 프로젝트에서 작업하는 경우 브라우저 환경에서만 유용한 “DOM” 라이브러리를 제외할 수 있습니다.

strict

“strict” 옵션은 더 강력한 검사를 활성화하여 타입 안전성을 높입니다. TypeScript 6.0부터 기본적으로 활성화됩니다. 그 이전 버전에서는 tsconfig.json에서 명시적으로 true로 설정해야 합니다. “strict”를 활성화하면 TypeScript는 다음을 수행합니다.

- 각 소스 파일에 “use strict”를 사용하여 코드를 내보냅니다.
- 타입 검사 과정에서 “null”과 “undefined”를 고려합니다.
- 타입 어노테이션이 없을 때 “any” 타입 사용을 비활성화합니다.
- 그렇지 않으면 “any” 타입을 의미하게 되는 “this” 표현식을 사용할 때 오류가 발생합니다.

module

“module” 프로퍼티는 컴파일된 프로그램에서 지원할 모듈 시스템을 설정합니다. 런타임에는 지정된 모듈 시스템에 따라 모듈 로더가 의존성을 찾고 실행하는 데 사용됩니다.

JavaScript에서 가장 일반적으로 사용하는 모듈 로더는 서버 측 애플리케이션을 위한 Node.js CommonJS와 브라우저 기반 웹 애플리케이션의 AMD 모듈을 위한 RequireJS입니다. TypeScript는 UMD, System, ESNext, ES2015/ES6 및 ES2020을 포함한 다양한 모듈 시스템용 코드를 내보낼 수 있습니다. 모듈 시스템은 대상 환경과 해당 환경에서 사용할 수 있는 모듈 로딩 메커니즘에 따라 선택해야 합니다.

참고: 이전 모듈 시스템(AMD, UMD, SystemJS)에 대한 지원은 TypeScript 6.0에서 더 이상 사용되지 않는 기능으로 지정되었으며 TypeScript 7.0에서는 더 이상 지원되지 않습니다.

moduleResolution

“moduleResolution” 프로퍼티는 모듈 해석 전략을 지정합니다. 최신 TypeScript 코드에는 “nodenext” 또는 “bundler”를 사용하세요. “classic” 전략은 이전 TypeScript 버전(1.6 이전)에서만 사용됩니다.

esModuleInterop

“esModuleInterop” 프로퍼티는 “default” 프로퍼티를 사용하여 내보내지 않은 CommonJS 모듈에서 기본 가져오기를 허용합니다. 이 프로퍼티는 내보낸 JavaScript에서 호환성을 보장하는 shim을 제공합니다. 이 옵션을 활성화하면 `import MyLibrary from "my-library"`를 `import * as MyLibrary from "my-library"` 대신 사용할 수 있습니다.

“esModuleInterop”은 호환성을 깨는 변경을 피하기 위해 원래 선택적으로 활성화했지만, 오랫동안 권장 기본값이었습니다. 이를 비활성화하

면 ESM과 CommonJS를 함께 사용할 때 미묘한 런타임 문제가 발생할 수 있습니다. 참고: TypeScript 6.0부터 이 더 안전한 상호 운용 동작은 항상 활성화됩니다.

TypeScript 6.0에서는 일부 오래된 구성 옵션과 구문 형식이 더 이상 사용되지 않는 기능으로 지정되거나 이전 동작을 거쳐 전환되었습니다. TypeScript 7.0에서는 하드 오류가 발생하거나 아무 동작도 하지 않습니다.

아무 동작도 하지 않으며 하드 오류로 전환된 더 이상 사용되지 않는 기능은 다음과 같습니다.

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`
- `esModuleInterop` 또는 `allowSyntheticDefaultImports` 비활성화
- `alwaysStrict` 비활성화
- 네임스페이스 선언의 `module` 키워드
- import의 `asserts`
- `/// <reference no-default-lib />`를 `skipDefaultLibCheck` 아래에서 사용
- `--ignoreConfig`를 사용하지 않는 경우 로컬 `tsconfig.json`이 있는 CLI 파일 경로

jsx

"jsx" 프로퍼티는 ReactJS에서 사용하는 .tsx 파일에만 적용되며 JSX 구문이 JavaScript로 컴파일되는 방식을 제어합니다. 일반적으로 사용

하는 옵션은 “preserve”로, JSX를 변경하지 않고 유지한 .jsx 파일로 컴파일하여 Babel과 같은 다른 도구에서 추가 변환을 수행할 수 있게 합니다.

skipLibCheck

“skipLibCheck” 프로퍼티를 사용하면 TypeScript가 가져온 타사 패키지 전체의 타입 검사를 수행하지 않습니다. 이 프로퍼티는 프로젝트의 컴파일 시간을 줄여 줍니다. TypeScript는 이러한 패키지가 제공하는 타입 정의에 대해서는 여전히 여러분의 코드를 검사합니다.

files

“files” 프로퍼티는 프로그램에 항상 포함해야 하는 파일 목록을 컴파일러에 지정합니다.

include

“include” 프로퍼티는 포함하려는 파일 목록을 컴파일러에 지정합니다. 이 프로퍼티에서는 모든 하위 디렉터리를 나타내는 “*”, *모든 파일 이름을 나타내는* “”, 선택적 문자를 나타내는 “?”와 같은 glob 형식의 패턴을 사용할 수 있습니다.

exclude

“exclude” 프로퍼티는 컴파일에 포함하지 않아야 하는 파일 목록을 컴파일러에 지정합니다. 여기에는 “node_modules” 또는 테스트 파일과 같은 파일이 포함될 수 있습니다. 참고: tsconfig.json에서는 주석을 사용할 수 있습니다.

importHelpers

TypeScript는 특정 고급 JavaScript 기능이나 이전 버전으로 다운레벨된 JavaScript 기능을 위한 코드를 생성할 때 헬퍼 코드를 사용합니다. 기본적으로 이러한 헬퍼는 이를 사용하는 파일마다 중복됩니다. `importHelpers` 옵션을 사용하면 이러한 헬퍼를 대신 `tslib` 모듈에서 가져와 JavaScript 출력을 더 효율적으로 만듭니다.

TypeScript 마이그레이션 조언

대규모 프로젝트에서는 처음에 TypeScript와 JavaScript 코드가 공존하는 점진적 전환 방식을 채택하는 것이 좋습니다. 소규모 프로젝트만 한 번에 TypeScript로 마이그레이션할 수 있습니다.

이 전환의 첫 번째 단계는 빌드 체인 프로세스에 TypeScript를 도입하는 것입니다. 기존 JavaScript 파일과 .ts 및 .tsx 파일이 공존할 수 있도록 하는 "allowJs" 컴파일러 옵션을 사용하면 됩니다. TypeScript가 JavaScript 파일에서 변수의 타입을 추론할 수 없을 때 "any" 타입으로 대체하므로, 마이그레이션 초기에는 컴파일러 옵션에서 "noImplicitAny"를 비활성화하는 것이 좋습니다.

두 번째 단계는 각 모듈을 변환하는 동안 테스트를 실행할 수 있도록 JavaScript 테스트가 TypeScript 파일과 함께 작동하는지 확인하는 것입니다. Jest를 사용한다면 Jest로 TypeScript 프로젝트를 테스트할 수 있게 해 주는 `ts-jest` 사용을 고려하세요.

세 번째 단계는 프로젝트에 타사 라이브러리의 타입 선언을 포함하는 것입니다. 이러한 선언은 라이브러리에 번들로 포함되어 있거나 DefinitelyTyped에서 찾을 수 있습니다. https://www.typescriptlang.org/dt/search에서 검색하고 다음 명령으로 설치할 수 있습니다.

```
npm install --save-dev @types/package-name
```

또는

```
yarn add --dev @types/package-name
```

네 번째 단계는 의존성 그래프의 리프부터 시작하여 상향식 접근 방식으로 모듈을 하나씩 마이그레이션하는 것입니다. 다른 모듈에 의존하지 않는 모듈부터 변환하는 것이 핵심입니다. 의존성 그래프를 시각화하려면 "madge" 도구를 사용할 수 있습니다.

이러한 초기 변환에 적합한 모듈은 유틸리티 함수와 외부 API 또는 명세와 관련된 코드입니다. 프로젝트에 포함할 TypeScript 타입 정의를 Swagger 계약, GraphQL 또는 JSON 스키마에서 자동으로 생성할 수 있습니다.

명세나 공식 스키마가 없다면 서버가 반환한 JSON과 같은 원시 데이터에서 타입을 생성할 수 있습니다. 그러나 누락되는 예외 사례를 방지하려면 데이터보다 명세에서 타입을 생성하는 것이 좋습니다.

마이그레이션 중에는 코드 리팩터링을 삼가고 모듈에 타입을 추가하는 데만 집중하세요.

다섯 번째 단계는 모든 타입을 알고 정의하도록 강제하여 프로젝트에서 더 나은 TypeScript 경험을 제공하는 "noImplicitAny"를 활성화하는 것입니다.

마이그레이션 중에는 JavaScript 파일에서 TypeScript 타입 검사를 활성화하는 `@ts-check` 지시어를 사용할 수 있습니다. 이 지시어는 느슨한 형태의 타입 검사를 제공하며 처음에는 JavaScript 파일의 문제를 식별하는 데 사용할 수 있습니다. 파일에 `@ts-check`가 포함되면 TypeScript는 JSDoc 스타일 주석을 사용하여 정의를 추론하려고 합니다. 다만 JSDoc

어노테이션은 마이그레이션의 아주 초기 단계에서만 사용하는 것을 고려하세요.

tsconfig.json에서 `noEmitOnError`의 기본값을 false로 유지하는 것을 고려하세요. 그러면 오류가 보고되더라도 JavaScript 소스 코드를 출력할 수 있습니다.

타입 시스템 살펴보기

TypeScript 언어 서비스

tsserver라고도 하는 TypeScript 언어 서비스는 오류 보고, 진단, 저장 시 컴파일, 이름 변경, 정의로 이동, 자동 완성 목록, 시그니처 도움말 등 다양한 기능을 제공합니다. 주로 통합 개발 환경(IDE)에서 IntelliSense 지원을 제공하는 데 사용됩니다. Visual Studio Code와 원활하게 통합되며 Conquer of Completion (Coc)과 같은 도구에서도 사용됩니다.

개발자는 전용 API를 활용하고 자체 언어 서비스 플러그인을 만들어 TypeScript 편집 환경을 개선할 수 있습니다. 이는 특별한 린팅 기능을 구현하거나 사용자 정의 템플릿 언어의 자동 완성을 활성화할 때 특히 유용합니다.

실제 사용자 정의 플러그인의 예로는 styled components의 CSS 프로퍼티에 대한 구문 오류 보고와 IntelliSense 지원을 제공하는 “typescript-styled-plugin”이 있습니다.

자세한 정보와 빠른 시작 가이드는 GitHub의 공식 TypeScript Wiki를 참고하세요. https://github.com/microsoft/TypeScript/wiki/

구조적 타이핑

TypeScript는 구조적 타입 시스템을 기반으로 합니다. 즉, C#이나 C와 같은 명목적 타입 시스템과 달리 타입의 호환성과 동등성은 이름이나 선언 위치가 아니라 타입의 실제 구조 또는 정의에 의해 결정됩니다.

TypeScript의 구조적 타입 시스템은 JavaScript의 동적 덕 타이핑 시스템이 런타임에 작동하는 방식을 바탕으로 설계되었습니다.

다음 예제는 유효한 TypeScript 코드입니다. 보이는 것처럼 "X"와 "Y"는 선언 이름이 서로 다르지만 동일한 멤버 "a"를 가집니다. 타입은 구조에 따라 결정되며, 이 경우 구조가 같으므로 서로 호환되고 유효합니다.

```
type X = {
    a: string;
};
type Y = {
    a: string;
};
const x: X = { a: 'a' };
const y: Y = x; // Valid
```

TypeScript의 기본 비교 규칙

TypeScript의 비교 과정은 재귀적으로 이루어지며 모든 깊이에 중첩된 타입에 대해 실행됩니다.

"Y"가 최소한 "X"와 동일한 멤버를 가지고 있으면 타입 "X"는 "Y"와 호환됩니다.

```
type X = {
    a: string;
};
```

```
const y = { a: 'A', b: 'B' }; // Valid, as it has at least the same members
as X
const r: X = y;
```

함수 매개변수는 이름이 아니라 타입으로 비교합니다.

```
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // Valid
x = y; // Valid
```

함수 반환 타입은 같아야 합니다.

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // Invalid
x = y; // Invalid
```

소스 함수의 반환 타입은 대상 함수 반환 타입의 하위 타입이어야 합니다.

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // Valid
y = x; // Invalid member b is missing
```

함수 매개변수를 버리는 것은 허용됩니다. 이는 JavaScript에서 흔한 관행으로, 예를 들면 “Array.prototype.map()”을 사용할 때 그렇습니다.

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

따라서 다음 타입 선언은 완전히 유효합니다.

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Missing b parameter
y = x; // Valid
```

소스 타입의 추가 선택적 매개변수는 모두 유효합니다.

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valid
x = y; //Valid
```

소스 타입에 대응하는 매개변수가 없는 대상 타입의 선택적 매개변수는 모두 유효하며 오류가 아닙니다.

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valid
x = y; // Valid
```

나머지 매개변수는 선택적 매개변수가 무한히 이어지는 것으로 취급됩니다.

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //valid
```

오버로드가 있는 함수는 오버로드 시그니처가 구현 시그니처와 호환되면 유효합니다.

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Valid
x('a', 1); // Valid

function y(a: string): void; // Invalid, not compatible with implementation
signature
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
}
y('a');
y('a', 1);
```

소스와 대상의 매개변수를 상위 타입이나 하위 타입에 할당할 수 있으면 함수 매개변수 비교가 성공합니다(이변성).

```
// Supertype
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// Subtype
class Y extends X {}
// Subtype
class Z extends X {}

type GetA = (x: X) => string;
const getA: GetA = x => x.a;

// Bivariance does accept supertypes
```

```
console.log(getA(new X('x'))); // Valid
console.log(getA(new Y('Y'))); // Valid
console.log(getA(new Z('z'))); // Valid
```

열거형은 숫자와 비교할 수 있으며 그 반대도 유효하지만, 서로 다른 열거형 타입의 열거형 값을 비교하는 것은 유효하지 않습니다.

```
enum X {
    A,
    B,
}
enum Y {
    A,
    B,
    C,
}
const xa: number = X.A; // Valid
const ya: Y = 0; // Valid
X.A === Y.A; // Invalid
```

클래스의 인스턴스는 private 멤버와 protected 멤버에 대한 호환성 검사를 받습니다.

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
        this.a = value;
    }
```

```
}

let x: X = new Y('y'); // Invalid
```

비교 검사는 서로 다른 상속 계층을 고려하지 않습니다. 예를 들면 다음과 같습니다.

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
class Y extends X {
    public a: string;
    constructor(value: string) {
        super(value);
        this.a = value;
    }
}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // Valid
x === z; // Valid even if z is from a different inheritance hierarchy
```

제네릭은 제네릭 매개변수를 적용한 뒤의 결과 타입을 바탕으로 구조를 사용해 비교하며, 최종 결과만 비제네릭 타입으로 비교합니다.

```ts
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Invalid as the type argument is used in the final structure

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // Valid as the type argument is not used in the final structure
```

제네릭의 타입 인수를 지정하지 않으면 지정되지 않은 모든 인수는 “any” 타입으로 취급됩니다.

```ts
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // Valid
```

기억하세요.

```ts
let a: number = 1;
let b: number = 2;
a = b; // Valid, everything is assignable to itself

let c: any;
c = 1; // Valid, all types are assignable to any

let d: unknown;
d = 1; // Valid, all types are assignable to unknown

let e: unknown;
let e1: unknown = e; // Valid, unknown is only assignable to itself and any
let e2: any = e; // Valid
```

```
let e3: number = e; // Invalid

let f: never;
f = 1; // Invalid, nothing is assignable to never

let g: void;
let g1: any;
g = 1; // Invalid, void is not assignable to or from anything except any
g = g1; // Valid
```

"strictNullChecks"가 활성화된 경우 "null"과 "undefined"는 "void"와 비슷하게 취급되며, 그렇지 않으면 "never"와 비슷하게 취급된다는 점에 유의하세요.

집합으로서의 타입

TypeScript에서 타입은 가능한 값의 집합입니다. 이 집합은 타입의 도메인이라고도 합니다. 타입의 각 값은 집합의 원소로 볼 수 있습니다. 타입은 집합의 각 원소가 해당 집합의 구성원으로 간주되기 위해 충족해야 하는 제약 조건을 설정합니다. TypeScript의 주된 작업은 한 집합이 다른 집합의 부분집합인지 검사하고 확인하는 것입니다.

TypeScript는 다양한 종류의 집합을 지원합니다.

집합 용어	TypeScript	참고
공집합	never	"never"는 자기 자신 외에는 아무것도 포함하지 않습니다
단일 원소 집합	undefined / null / literal type	
유한 집합	boolean / union	

무한 집합	string / number / object	
전체집합	any / unknown	모든 원소는 “any”의 구성원이고 모든 집합은 그 부분집합입니다 / “unknown”은 “any”의 타입 안전한 대응 타입입니다

다음은 몇 가지 예입니다.

TypeScript	집합 용어	예
never	∅ (공집합)	const x: never = ‘x’; // Error: Type ‘string’ is not assignable to type ‘never’
리터럴 타입	단일 원소 집합	type X = ‘X’; type Y = 7;
T에 할당 가능한 값	값 ∈ T (구성원)	type XY = ‘X’ | ‘Y’; const x: XY = ‘X’;
T2에 할당 가능한 T1	T1 ⊆ T2 (부분집합)	type XY = ‘X’ | ‘Y’; const x: XY = ‘X’; const j: XY = ‘J’; // Type ‘“J”’ is not assignable to type ‘XY’.
T1 extends T2	T1 ⊆ T2 (부분집합)	type X = ‘X’ extends string ? true : false;
T1 | T2	T1 ∪ T2 (합집합)	type XY = ‘X’ | ‘Y’; type JK = 1 | 2;
T1 & T2	T1 ∩ T2 (교집합)	type X = { a: string } type Y = { b: string } type XY = X & Y const x: XY = { a: ‘a’, b: ‘b’ }
unknown	전체집합	const x: unknown = 1

합집합 (T1 | T2)는 더 넓은 집합(둘 다)을 만듭니다.

```
type X = {
    a: string;
};
type Y = {
    b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Valid
```

교집합 (T1 & T2)는 더 좁은 집합(공유되는 부분만)을 만듭니다.

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // Invalid
const j: XY = { a: 'a', b: 'b' }; // Valid
```

이 맥락에서 extends 키워드는 "부분집합"으로 생각할 수 있습니다. 이는 타입에 제약 조건을 설정합니다. extends를 제네릭과 함께 사용하면 제네릭 타입 매개변수를 더 구체적인 타입으로 제한합니다.

여기서 extends는 OOP 의미의 클래스 상속과 아무런 관련이 없다는 점에 유의하세요.

TypeScript는 구조적 타입을 사용하며 엄격한 명목적 계층 구조가 없습니다. 실제로 아래 예제처럼 TypeScript는 객체의 구조 또는 형태를 고

려하므로 두 타입 중 어느 쪽도 다른 쪽의 하위 타입이 아니면서 서로 겹칠 수 있습니다.

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
interface Z extends Y {
    c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
    a: string;
    b: string;
    c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // Valid
```

타입 지정하기: 타입 선언과 타입 단언

TypeScript에서는 여러 가지 방법으로 타입을 할당할 수 있습니다.

타입 선언

다음 예제에서는 x: X (“: Type”)를 사용하여 변수 x의 타입을 선언합니다.

```
type X = {
    a: string;
};

// Type declaration
const x: X = {
    a: 'a',
};
```

변수가 지정된 형식에 맞지 않으면 TypeScript가 오류를 보고합니다. 예를 들면 다음과 같습니다.

```
type X = {
    a: string;
};

const x: X = {
    a: 'a',
    b: 'b', // Error: Object literal may only specify known properties
};
```

타입 단언

as 키워드를 사용하여 단언을 추가할 수 있습니다. 이는 개발자가 타입에 관해 더 많은 정보를 가지고 있음을 컴파일러에 알리고 발생할 수 있는 모든 오류를 표시하지 않게 합니다.

예를 들면 다음과 같습니다.

```
type X = {
    a: string;
```

```
};
const x = {
    a: 'a',
    b: 'b',
} as X;
```

위 예제에서는 as 키워드를 사용하여 객체 x가 타입 X를 갖는다고 단언합니다. 이는 객체에 타입 정의에 없는 추가 프로퍼티 b가 있더라도 지정된 타입을 준수한다고 TypeScript 컴파일러에 알립니다.

타입 단언은 특히 DOM을 다룰 때처럼 더 구체적인 타입을 지정해야 하는 상황에 유용합니다. 예를 들면 다음과 같습니다.

```
const myInput = document.getElementById('my_input') as HTMLInputElement;
```

여기서 타입 단언 as HTMLInputElement는 getElementById의 결과를 HTMLInputElement로 취급해야 한다고 TypeScript에 알리는 데 사용됩니다. 타입 단언은 아래의 템플릿 리터럴 예제처럼 키를 다시 매핑하는 데도 사용할 수 있습니다.

```
type J<Type> = {
    [Property in keyof Type as `prefix_${string &
        Property}`]: () => Type[Property];
};
type X = {
    a: string;
    b: number;
};
type Y = J<X>;
```

이 예제에서 J<Type> 타입은 템플릿 리터럴이 포함된 매핑된 타입을 사용하여 Type의 키를 다시 매핑합니다. 각 키에 “prefix_”가 추가된 새 프로퍼티를 만들고, 해당 값은 원래 프로퍼티 값을 반환하는 함수입니다.

타입 단언을 사용할 때 TypeScript가 초과 프로퍼티 검사를 실행하지 않는다는 점에 유의할 필요가 있습니다. 따라서 객체의 구조를 미리 알고 있다면 일반적으로 타입 선언을 사용하는 것이 좋습니다.

앰비언트 선언

앰비언트 선언은 JavaScript 코드의 타입을 설명하는 파일이며 파일 이름 형식은 `.d.ts.`입니다. 보통 기존 JavaScript 라이브러리에 타입 어노테이션을 추가하거나 프로젝트의 기존 JS 파일에 타입을 추가하기 위해 가져와서 사용합니다.

많이 사용되는 라이브러리의 타입은 다음에서 찾을 수 있습니다. https://github.com/DefinitelyTyped/DefinitelyTyped/

그리고 다음 명령으로 설치할 수 있습니다.

```
npm install --save-dev @types/library-name
```

직접 정의한 앰비언트 선언은 "트리플 슬래시" 참조를 사용해 가져올 수 있습니다.

```
/// <reference path="./library-types.d.ts" />
```

// `@ts-check`를 사용하면 JavaScript 파일 안에서도 앰비언트 선언을 사용할 수 있습니다.

`declare` 키워드는 기존 JavaScript 코드를 가져오지 않고도 타입을 정의할 수 있게 하며, 다른 파일 또는 전역에 있는 타입을 위한 자리 표시자 역할을 합니다.

프로퍼티 검사와 초과 프로퍼티 검사

TypeScript는 구조적 타입 시스템을 기반으로 하지만, 초과 프로퍼티 검사는 객체가 타입에 지정된 프로퍼티만 정확히 가지는지 확인하는 TypeScript 기능입니다.

초과 프로퍼티 검사는 객체 리터럴을 변수에 할당하거나 함수의 초과 프로퍼티에 인수로 전달할 때 수행됩니다. 예를 들면 다음과 같습니다.

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
const x: X = y; // Valid because structural typing
const w: X = { a: 'a', b: 'b' }; // Invalid because excess property checking
```

약한 타입

모든 프로퍼티가 선택적인 프로퍼티 집합으로만 이루어진 타입은 약한 타입으로 간주됩니다.

```
type X = {
    a?: string;
    b?: string;
};
```

TypeScript에서는 겹치는 부분이 없을 때 약한 타입에 어떤 값을 할당하는 것을 오류로 간주합니다. 예를 들어 다음 코드는 오류를 발생시킵니다.

```
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
```

```
fn({ c: 'c' }); // Invalid
```

권장되지는 않지만 필요하다면 타입 단언을 사용하여 이 검사를 우회할 수 있습니다.

```
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Valid
```

또는 약한 타입의 인덱스 시그니처에 unknown을 추가할 수 있습니다.

```
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Valid
```

엄격한 객체 리터럴 검사(Freshness)

엄격한 객체 리터럴 검사는 “freshness”라고도 하며, 일반적인 구조적 타입 검사에서는 알아차리지 못할 수 있는 초과 프로퍼티나 철자가 잘못된 프로퍼티를 찾는 데 도움이 되는 TypeScript 기능입니다.

객체 리터럴을 만들 때 TypeScript 컴파일러는 이를 “fresh”한 것으로 간주합니다. 객체 리터럴을 변수에 할당하거나 매개변수로 전달할 때, 객체 리터럴에 대상 타입에 없는 프로퍼티가 지정되어 있으면 TypeScript가 오류를 발생시킵니다.

그러나 객체 리터럴이 확장되거나 타입 단언을 사용하면 “freshness”가 사라집니다.

다음은 이를 보여 주는 몇 가지 예제입니다.

```
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Freshness check: Invalid assignment
var y: Y;
y = { a: 'a', bx: 'bx' }; // Freshness check: Invalid assignment

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Widening: No errors, structurally type compatible

fn({ a: 'a', bx: 'b' }); // Freshness check: Invalid argument

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // Widening: No Freshness check
```

타입 추론

TypeScript는 다음 상황에서 어노테이션이 제공되지 않아도 타입을 추론할 수 있습니다.

- 변수 초기화.
- 멤버 초기화.
- 매개변수의 기본값 설정.
- 함수 반환 타입.

예를 들면 다음과 같습니다.

```
let x = 'x'; // The type inferred is string
```

TypeScript 컴파일러는 값이나 표현식을 분석하고 사용 가능한 정보를 바탕으로 타입을 결정합니다.

더 고급 추론

타입 추론에 여러 표현식이 사용되면 TypeScript는 "최적 공통 타입"을 찾습니다. 예를 들면 다음과 같습니다.

```
let x = [1, 'x', 1, null]; // The type inferred is: (string | number | null)
[]
```

컴파일러가 최적 공통 타입을 찾을 수 없으면 유니온 타입을 반환합니다. 예를 들면 다음과 같습니다.

```
let x = [new RegExp('x'), new Date()]; // Type inferred is: (RegExp | Date)
[]
```

TypeScript는 변수의 위치를 바탕으로 "문맥적 타이핑"을 사용하여 타입을 추론합니다. 다음 예제에서 컴파일러는 다양한 일반 JavaScript 구문과 DOM의 앰비언트 선언이 들어 있는 lib.d.ts 파일에 정의된 `click` 이벤트 타입을 바탕으로 `e`가 `MouseEvent` 타입임을 압니다.

```
window.addEventListener('click', function (e) {}); // The inferred type of e
is MouseEvent
```

타입 넓히기

타입 넓히기는 TypeScript가 타입 어노테이션 없이 초기화된 변수에 타입을 할당하는 과정입니다. 좁은 타입에서 더 넓은 타입으로 넓히는 것은 허용하지만 그 반대는 허용하지 않습니다. 다음 예제를 살펴보세요.

```
let x = 'x'; // TypeScript infers as string, a wide type
let y: 'y' | 'x' = 'y'; // y types is a union of literal types
y = x; // Invalid Type 'string' is not assignable to type '"x" | "y"'.
```

TypeScript는 초기화할 때 제공된 단일 값(x)을 바탕으로 x에 string을 할당하며, 이는 타입 넓히기의 예입니다.

TypeScript는 예를 들어 “const”를 사용하여 타입 넓히기 과정을 제어하는 방법을 제공합니다.

Const

변수를 선언할 때 const 키워드를 사용하면 TypeScript가 더 좁은 타입을 추론합니다.

예를 들면 다음과 같습니다.

```
const x = 'x'; // TypeScript infers the type of x as 'x', a narrower type
let y: 'y' | 'x' = 'y';
y = x; // Valid: The type of x is inferred as 'x'
```

const를 사용해 변수 x를 선언하면 타입이 특정 리터럴 값 ’x’로 좁혀집니다. x의 타입이 좁혀졌으므로 오류 없이 변수 y에 할당할 수 있습니다. 타입을 추론할 수 있는 이유는 const 변수를 재할당할 수 없기 때문입니다. 따라서 타입을 특정 리터럴 타입, 이 경우에는 리터럴 타입 ’x’로 좁힐 수 있습니다.

타입 매개변수의 Const 수정자

TypeScript 버전 5.0부터 제네릭 타입 매개변수에 const 속성을 지정할 수 있습니다. 이를 통해 가능한 한 가장 정밀한 타입을 추론할 수 있습니다. const를 사용하지 않은 예제를 살펴보겠습니다.

```
function identity<T>(value: T) {
    // No const here
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Type inferred is: { a: string; b: string; }
```

보이는 것처럼 프로퍼티 a와 b는 string 타입으로 추론됩니다.

이제 const 버전과의 차이를 살펴보겠습니다.

```
function identity<const T>(value: T) {
    // Using const modifier on type parameters
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Type inferred is: { a: "a"; b: "b"; }
```

이제 프로퍼티 a와 b가 단순한 string 타입이 아니라 문자열 리터럴로 추론되는 것을 확인할 수 있습니다.

Const 단언

이 기능을 사용하면 초기화 값을 바탕으로 더 정밀한 리터럴 타입을 갖는 변수를 선언하여, 해당 값을 불변 리터럴로 취급해야 함을 컴파일러에 알릴 수 있습니다. 다음은 몇 가지 예제입니다.

단일 프로퍼티에 적용하는 경우:

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

객체 전체에 적용하는 경우:

```
const v = {
    x: 1,
    y: 2,
} as const;
```

이는 튜플의 타입을 정의할 때 특히 유용할 수 있습니다.

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // Tuple of readonly [1, 2, 3]
```

명시적 타입 어노테이션

타입을 구체적으로 지정하여 전달할 수 있습니다. 다음 예제에서 프로퍼티 x는 number 타입입니다.

```
const v = {
    x: 1, // Inferred type: number (widening)
};
v.x = 3; // Valid
```

리터럴 타입의 유니온을 사용하면 타입 어노테이션을 더 구체적으로 만들 수 있습니다.

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // x is now a union of literal types: 1 | 2 | 3
};
v.x = 3; // Valid
v.x = 100; // Invalid
```

타입 좁히기

타입 좁히기는 일반적인 타입이 더 구체적인 타입으로 좁혀지는 TypeScript의 과정입니다. TypeScript가 코드를 분석하여 특정 조건이나 연산이 타입 정보를 구체화할 수 있다고 판단할 때 발생합니다.

타입 좁히기는 다음을 비롯해 다양한 방식으로 이루어질 수 있습니다.

조건

if나 switch 같은 조건문을 사용하면 TypeScript가 조건의 결과를 바탕으로 타입을 좁힐 수 있습니다. 예를 들면 다음과 같습니다.

```
let x: number | undefined = 10;

if (x !== undefined) {
      x += 100; // The type is number, which had been narrowed by the condition
}
```

예외 발생 또는 반환

오류를 발생시키거나 분기에서 일찍 반환하면 TypeScript가 타입을 좁히는 데 도움이 될 수 있습니다. 예를 들면 다음과 같습니다.

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

TypeScript에서 타입을 좁히는 다른 방법은 다음과 같습니다.

- instanceof 연산자: 객체가 특정 클래스의 인스턴스인지 확인하는 데 사용됩니다.
- in 연산자: 객체에 프로퍼티가 존재하는지 확인하는 데 사용됩니다.
- typeof 연산자: 런타임에 값의 타입을 확인하는 데 사용됩니다.

- `Array.isArray()` 같은 기본 제공 함수: 값이 배열인지 확인하는 데 사용됩니다.

판별 유니온

"판별 유니온(Discriminated Union)"을 사용하는 것은 유니온에 포함된 서로 다른 타입을 구별하기 위해 객체에 명시적인 "태그(tag)"를 추가하는 TypeScript 패턴입니다. 이 패턴은 "태그된 유니온(tagged union)"이라고도 합니다. 다음 예제에서는 "태그"를 "type" 프로퍼티로 나타냅니다.

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // type is A
        case 'type_b':
            return input.value + 'extra'; // type is B
    }
};
```

사용자 정의 타입 가드

TypeScript가 타입을 판별할 수 없는 경우에는 "사용자 정의 타입 가드(user-defined type guard)"라는 헬퍼 함수를 작성할 수 있습니다. 다음 예제에서는 특정 필터링을 적용한 뒤 타입을 좁히기 위해 타입 술어(Type Predicate)를 사용합니다.

```
const data = ['a', null, 'c', 'd', null, 'f'];
```

```
const r1 = data.filter(x => x != null); // The type is (string | null)[], TypeScript was not able to infer the type properly

const isValid = (item: string | null): item is string => item !== null; // Custom type guard

const r2 = data.filter(isValid); // The type is fine now string[], by using the predicate type guard we were able to narrow the type
```

Switch-true 좁히기

TypeScript 5.3에는 switch-true 좁히기가 추가되어, 복잡한 if/else 체인을 불리언 조건을 사용하는 switch (true)로 대체할 수 있습니다. 가독성이 향상되면서도 타입은 계속 좁혀집니다. 패턴 매칭과 비슷하지만 더 간단합니다.

```
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}
```

원시 타입

TypeScript는 7가지 원시 타입을 지원합니다. 원시 데이터 타입은 객체가 아니며 관련된 메서드가 없는 타입을 의미합니다. TypeScript에서 모든 원시 타입은 불변이므로 값이 할당된 후에는 변경할 수 없습니다.

string

string 원시 타입은 텍스트 데이터를 저장하며, 값은 항상 큰따옴표나 작은따옴표로 묶입니다.

```
const x: string = 'x';
const y: string = 'y';
```

문자열을 백틱(`) 문자로 감싸면 여러 줄에 걸쳐 작성할 수 있습니다.

```
let sentence: string = `xxx,
   yyy`;
```

boolean

TypeScript의 boolean 데이터 타입은 true 또는 false인 이진 값을 저장합니다.

```
const isReady: boolean = true;
```

number

TypeScript의 number 데이터 타입은 64비트 부동 소수점 값으로 표현됩니다. number 타입은 정수와 소수를 나타낼 수 있습니다. TypeScript는 16진수, 2진수, 8진수도 지원합니다. 예를 들면 다음과 같습니다.

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // Hexadecimal starts with 0x
const binary: number = 0b1010; // Binary starts with 0b
const octal: number = 0o633; // Octal starts with 0o
```

bigint

bigint는 number가 지원하는 최대 안전 정수인 2^53 - 1보다 큰 정숫값을 나타냅니다.

bigint는 내장 함수 BigInt()를 호출하거나 정수 숫자 리터럴 끝에 n을 추가하여 만들 수 있습니다.

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

참고:

- bigint 값은 number와 혼합할 수 없으며 내장 Math와 함께 사용할 수 없습니다. 같은 타입으로 강제 변환해야 합니다.
- bigint 값은 target 구성이 ES2020 이상인 경우에만 사용할 수 있습니다.

Symbol

심볼은 객체의 프로퍼티 키로 사용할 수 있는 고유 식별자로, 이름 충돌을 방지합니다.

```
type Obj = {
    [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null과 undefined

null과 undefined 타입은 모두 값이 없거나 어떤 값도 존재하지 않음을 나타냅니다.

undefined 타입은 값이 할당 또는 초기화되지 않았음을 의미하거나 의도하지 않은 값의 부재를 나타냅니다.

null 타입은 필드에 값이 없다는 사실을 알고 있어 해당 값을 사용할 수 없음을 의미하며, 의도적인 값의 부재를 나타냅니다.

Array

array는 같은 타입 또는 서로 다른 타입의 여러 값을 저장할 수 있는 데이터 타입입니다. 다음 구문을 사용하여 정의할 수 있습니다.

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // Union
```

TypeScript는 다음 구문을 사용하여 읽기 전용 배열을 지원합니다.

```
const x: readonly string[] = ['a', 'b']; // Readonly modifier
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Invalid
```

TypeScript는 튜플과 읽기 전용 튜플을 지원합니다.

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

any 데이터 타입은 문자 그대로 "모든" 값을 나타내며, TypeScript가 타입을 추론할 수 없거나 타입이 지정되지 않았을 때의 기본값입니다.

any를 사용하면 TypeScript 컴파일러가 타입 검사를 건너뛰므로 any를 사용하는 동안에는 타입 안전성이 보장되지 않습니다. 일반적으로 오류가 발생했을 때 컴파일러를 조용하게 만들기 위해 any를 사용하지 말고 오류를 수정하는 데 집중하세요. any를 사용하면 계약을 깨뜨리고 TypeScript 자동 완성의 이점을 잃을 수 있기 때문입니다.

any 타입은 컴파일러 오류를 억제할 수 있으므로 JavaScript에서 TypeScript로 점진적으로 마이그레이션할 때 유용할 수 있습니다.

새 프로젝트에서는 TypeScript 구성 옵션 noImplicitAny를 사용하세요. 이 옵션을 사용하면 any가 사용되거나 추론되는 곳에서 TypeScript가 오류를 발생시킵니다.

any 타입은 실제 타입 문제를 가릴 수 있는 오류의 원인이 되는 경우가 많습니다. 가능한 한 사용을 피하세요.

타입 어노테이션

var, let, const를 사용하여 선언한 변수에는 선택적으로 타입을 추가할 수 있습니다.

```
const x: number = 1;
```

TypeScript는 특히 단순한 타입을 잘 추론하므로 대부분의 경우 이러한 선언은 필요하지 않습니다.

함수에서는 매개변수에 타입 어노테이션을 추가할 수 있습니다.

```
function sum(a: number, b: number) {
    return a + b;
}
```

다음은 익명 함수(람다 함수라고도 함)를 사용하는 예제입니다.

```
const sum = (a: number, b: number) => a + b;
```

매개변수에 기본값이 있으면 이러한 어노테이션을 생략할 수 있습니다.

```
const sum = (a = 10, b: number) => a + b;
```

함수에 반환 타입 어노테이션을 추가할 수 있습니다.

```
const sum = (a = 10, b: number): number => a + b;
```

이는 특히 더 복잡한 함수에서 유용합니다. 구현 전에 반환 타입을 작성하면 함수를 구상하는 데 도움이 될 수 있기 때문입니다.

일반적으로 타입 시그니처에는 어노테이션을 추가하되 함수 본문의 지역 변수에는 추가하지 않는 것을 고려하고, 객체 리터럴에는 항상 타입을 추가하세요.

선택적 프로퍼티

객체는 프로퍼티 이름 끝에 물음표 ?를 추가하여 선택적 프로퍼티를 지정할 수 있습니다.

```
type X = {
    a: number;
    b?: number; // Optional
};
```

프로퍼티가 선택 사항인 경우 기본값을 지정할 수 있습니다.

```
type X = {
    a: number;
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

읽기 전용 프로퍼티

readonly 수정자를 사용하면 프로퍼티에 값을 쓰지 못하게 할 수 있습니다. 이 수정자는 프로퍼티를 다시 쓸 수 없도록 보장하지만 완전한 불변성을 보장하지는 않습니다.

```
interface Y {
    readonly a: number;
}

type X = {
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;

type K = {
    readonly [index: number]: string;
};
```

인덱스 시그니처

TypeScript에서는 string, number, symbol을 인덱스 시그니처로 사용할 수 있습니다.

```ts
type K = {
    [name: string | number]: string;
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
console.log(k[1]);
console.log(k['1']); // Same result as k[1]
```

JavaScript는 number 인덱스를 자동으로 string 인덱스로 변환하므로 k[1]과 k["1"]은 같은 값을 반환합니다.

타입 확장

interface를 확장할 수 있습니다(다른 타입에서 멤버를 복사합니다).

```ts
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
```

여러 타입을 확장하는 것도 가능합니다.

```ts
interface A {
    a: string;
}
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

extends 키워드는 인터페이스와 클래스에서만 작동하며, 타입에는 인터섹션을 사용합니다.

```
type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;
```

인터페이스를 사용하여 타입을 확장할 수 있지만 그 반대는 불가능합니다.

```
type A = {
    a: string;
};
interface B extends A {
    b: string;
}
```

리터럴 타입

리터럴 타입은 집합 타입 내에서 하나의 요소만 포함하는 집합으로, JavaScript 원시 값 하나를 정확하게 정의합니다.

TypeScript의 리터럴 타입에는 숫자, 문자열, 불리언이 있습니다.

리터럴의 예제는 다음과 같습니다.

```
const a = 'a'; // String literal type
const b = 1; // Numeric literal type
const c = true; // Boolean literal type
```

문자열, 숫자, 불리언 리터럴 타입은 유니온, 타입 가드, 타입 별칭에서 사용됩니다. 다음 예제에서는 유니온 타입 별칭을 확인할 수 있습니다. `O`는 지정된 값으로만 구성되며 다른 문자열은 유효하지 않습니다.

```
type O = 'a' | 'b' | 'c';
```

리터럴 추론

리터럴 추론은 변수나 매개변수의 값을 바탕으로 타입을 추론할 수 있게 해주는 TypeScript 기능입니다.

다음 예제에서 TypeScript는 `x`의 값을 나중에 변경할 수 없으므로 이를 리터럴 타입으로 간주하는 반면, `y`는 나중에 언제든지 수정할 수 있으므로 string으로 추론합니다.

```
const x = 'x'; // Literal type of 'x', because this value cannot be changed
let y = 'y'; // Type string, as we can change this value
```

다음 예제에서 TypeScript는 값이 나중에 언제든지 변경될 수 있다고 간주하므로 `o.x`를 `string`으로(`a` 리터럴이 아니라) 추론합니다.

```
type X = 'a' | 'b';

let o = {
    x: 'a', // This is a wider string
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x)); // Argument of type 'string' is not assignable to
parameter of type 'X'
```

보이는 것처럼 X가 더 좁은 타입이므로 `o.x`를 `fn`에 전달할 때 코드에서 오류가 발생합니다.

const 또는 X 타입으로 타입 단언을 사용하여 이 문제를 해결할 수 있습니다.

```
let o = {
    x: 'a' as const,
};
```

또는 다음과 같이 작성합니다.

```
let o = {
    x: 'a' as X,
};
```

strictNullChecks

strictNullChecks는 엄격한 null 검사를 적용하는 TypeScript 컴파일러 옵션입니다. 이 옵션을 활성화하면 유니온 타입 null | undefined를 사용해 해당 타입으로 명시적으로 선언한 변수와 매개변수에만 null 또는 undefined를 할당할 수 있습니다. 변수나 매개변수를 nullable로 명시적으로 선언하지 않으면 잠재적인 런타임 오류를 방지하기 위해 TypeScript가 오류를 생성합니다.

열거형

TypeScript에서 enum은 이름이 지정된 상숫값의 집합입니다.

```
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
    Blue = '#0000ff',
}
```

열거형은 다양한 방식으로 정의할 수 있습니다.

숫자 열거형

TypeScript에서 숫자 열거형은 각 상수에 숫자 값이 할당되는 열거형으로, 기본적으로 0부터 시작합니다.

```
enum Size {
    Small, // value starts from 0
    Medium,
    Large,
}
```

값을 명시적으로 할당하여 사용자 지정 값을 지정할 수 있습니다.

```
enum Size {
    Small = 10,
    Medium,
    Large,
}
console.log(Size.Medium); // 11
```

문자열 열거형

TypeScript에서 문자열 열거형은 각 상수에 문자열 값이 할당되는 열거형입니다.

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

참고: TypeScript에서는 문자열 멤버와 숫자 멤버가 함께 존재할 수 있는 이종 열거형을 사용할 수 있습니다.

상수 열거형

TypeScript의 상수 열거형은 모든 값을 컴파일 시간에 알 수 있고 열거형이 사용되는 모든 위치에 인라인되어 더 효율적인 코드를 만드는 특별한 종류의 열거형입니다.

```
const enum Language {
    English = 'EN',
    Spanish = 'ES',
}
console.log(Language.English);
```

다음과 같이 컴파일됩니다.

```
console.log('EN' /* Language.English */);
```

참고: 상수 열거형은 값이 하드코딩되어 열거형 자체가 제거되므로 자체 포함 라이브러리에서는 더 효율적일 수 있지만 일반적으로는 바람직하지 않습니다. 또한 상수 열거형에는 계산된 멤버를 포함할 수 없습니다.

역방향 매핑

TypeScript에서 열거형의 역방향 매핑은 값으로 열거형 멤버 이름을 가져오는 기능을 의미합니다. 기본적으로 열거형 멤버에는 이름에서 값으로 향하는 정방향 매핑이 있지만, 각 멤버의 값을 명시적으로 설정하면 역방향 매핑을 만들 수 있습니다. 역방향 매핑은 값으로 열거형 멤버를 찾거나 모든 열거형 멤버를 순회해야 할 때 유용합니다. 숫자 열거형 멤버만 역방향 매핑을 생성하며, 문자열 열거형 멤버에는 역방향 매핑이 전혀 생성되지 않는다는 점에 유의하세요.

다음 열거형은:

```
enum Grade {
    A = 90,
    B = 80,
```

```
    C = 70,
    F = 'fail',
}
```

다음과 같이 컴파일됩니다.

```
'use strict';
var Grade;
(function (Grade) {
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

따라서 값을 키에 매핑하는 것은 숫자 열거형 멤버에서는 작동하지만 문자열 열거형 멤버에서는 작동하지 않습니다.

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A

const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]); // Element implicitly has an 'any' type
because index expression is not of type 'number'.
```

앰비언트 열거형

TypeScript의 앰비언트 열거형은 관련 구현 없이 선언 파일(*.d.ts)에 정의되는 열거형의 한 종류입니다. 각 파일에서 구현 세부 정보를 가져올 필요 없이 여러 파일에서 타입 안전하게 사용할 수 있는 이름이 지정된 상수 집합을 정의할 수 있습니다.

계산된 멤버와 상수 멤버

TypeScript에서 계산된 멤버는 런타임에 값이 계산되는 열거형 멤버이고, 상수 멤버는 컴파일 시간에 값이 설정되어 런타임에 변경할 수 없는 멤버입니다. 계산된 멤버는 일반 열거형에서 사용할 수 있으며, 상수 멤버는 일반 열거형과 const 열거형 모두에서 사용할 수 있습니다.

```
// Constant members
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6 generation at compilation time

// Computed members
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // random number generated at run time
```

열거형은 멤버 타입으로 구성된 유니온으로 표현됩니다. 각 멤버의 값은 상수 표현식이나 비상수 표현식을 통해 결정할 수 있으며, 상숫값을 갖는 멤버에는 리터럴 타입이 할당됩니다. 예를 들어 타입 E와 그 하위 타입 E.A, E.B, E.C의 선언을 살펴보세요. 이 경우 E는 유니온 E.A | E.B | E.C를 나타냅니다.

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // Numeric literal
    B = 'bar', // String literal
    C = identity(42), // Opaque computed
}

console.log(E.C); //42
```

좁히기

TypeScript 좁히기는 조건부 블록 내에서 변수의 타입을 구체화하는 과정입니다. 변수가 둘 이상의 타입을 가질 수 있는 유니온 타입을 다룰 때 유용합니다.

TypeScript는 타입을 좁히는 여러 가지 방법을 인식합니다.

typeof 타입 가드

typeof 타입 가드는 내장 JavaScript 타입을 바탕으로 변수의 타입을 확인하는 TypeScript의 특정 타입 가드입니다.

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // x is number
    }
    return -1;
};
```

참 같은 값에 의한 좁히기

TypeScript의 참 같은 값 좁히기는 변수가 참 같은 값인지 거짓 같은 값인지 확인하여 그에 따라 타입을 좁힙니다.

```
const toUpperCase = (name: string | null) => {
    if (name) {
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

동등성 좁히기

TypeScript의 동등성 좁히기는 변수가 특정 값과 같은지 여부를 확인하여 그에 따라 타입을 좁힙니다.

타입을 좁히기 위해 switch 문 및 ===, !==, ==, != 같은 동등 연산자와 함께 사용합니다.

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

In 연산자 좁히기

TypeScript의 in 연산자 좁히기는 변수의 타입에 특정 프로퍼티가 존재하는지를 바탕으로 변수의 타입을 좁히는 방법입니다.

```
type Dog = {
    name: string;
    breed: string;
};
```

```
type Cat = {
    name: string;
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
    } else {
        return 'cat';
    }
};
```

instanceof 좁히기

TypeScript의 instanceof 연산자 좁히기는 객체가 특정 클래스나 인터페이스의 인스턴스인지 확인하여 생성자 함수를 바탕으로 변수의 타입을 좁히는 방법입니다.

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
    ) {}
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
```

```
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

할당

할당을 통한 TypeScript 좁히기는 변수에 할당된 값을 바탕으로 변수의 타입을 좁히는 방법입니다. 변수에 값이 할당되면 TypeScript는 할당된 값을 바탕으로 타입을 추론하고 추론된 타입과 일치하도록 변수의 타입을 좁힙니다.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

제어 흐름 분석

TypeScript의 제어 흐름 분석은 변수의 타입을 추론하기 위해 코드 흐름을 정적으로 분석하는 방법으로, 컴파일러가 분석 결과를 바탕으로 필요에 따라 해당 변수의 타입을 좁힐 수 있게 해줍니다.

TypeScript 4.4 이전에는 코드 흐름 분석이 if 문 안의 코드에만 적용되었지만, TypeScript 4.4부터는 const 변수를 통해 간접적으로 참조되는 조건식과 판별 프로퍼티 접근에도 적용할 수 있습니다.

예를 들면 다음과 같습니다.

```ts
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
    obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar: number }
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
        obj.foo;
    } else {
        obj.bar;
    }
};
```

좁히기가 발생하지 않는 몇 가지 예제는 다음과 같습니다.

```ts
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
        x.length; // Error, no narrowing because isString it is not const
    }
};

const f6 = (
    obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar: number }
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
        obj.foo; // Error, no narrowing because obj is assigned in function
body
```

```
    }
};
```

참고: 조건식에서는 최대 5단계의 간접 참조가 분석됩니다.

타입 술어

TypeScript의 타입 술어는 불리언 값을 반환하고 변수의 타입을 더 구체적인 타입으로 좁히는 데 사용되는 함수입니다.

```
const isString = (value: unknown): value is string => typeof value === 'string';

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('not a string');
    }
};
```

TypeScript 5.5는 타입 술어(예: `x is T`)를 `.filter` 같은 함수에서 자동으로 추론하므로 undefined 같은 값이 제거되는 시점을 인식하여 더 정확한 타입과 더 적은 오류를 제공합니다. 이는 명확한 검사(예: `x !== undefined`)에서는 작동하지만 `!!x`처럼 모호한 검사에서는 작동하지 않습니다.

```
const nums = [1, null, 2].filter(x => x !== null);
```

판별 유니온

TypeScript의 판별 유니온은 판별자라고 하는 공통 프로퍼티를 사용하여 유니온에서 가능한 타입의 집합을 좁히는 유니온 타입입니다.

```ts
type Square = {
    kind: 'square'; // Discriminant
    size: number;
};

type Circle = {
    kind: 'circle'; // Discriminant
    radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
    }
};

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

never 타입에 대하여

변수의 타입이 어떤 값도 포함할 수 없는 타입으로 좁혀지면 TypeScript 컴파일러는 해당 변수가 never 타입이어야 한다고 추론합니다. never 타입은 절대 생성될 수 없는 값을 나타내기 때문입니다.

```ts
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
```

```
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
        // val has type never here because it can never be anything other
than a string or a number
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
    }
};
```

완전성 검사

완전성 검사는 판별 유니온에서 가능한 모든 경우가 switch 문이나 if 문에서 처리되도록 보장하는 TypeScript 기능입니다.

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
            console.log('Moving up');
            break;
        case 'down':
            console.log('Moving down');
            break;
        default:
            const exhaustiveCheck: never = direction;
                console.log(exhaustiveCheck); // This line will never be
executed
    }
};
```

never 타입은 default 절이 모든 경우를 처리하도록 보장하며, switch 문에서 처리하지 않은 새 값이 Direction 타입에 추가되면 TypeScript

가 오류를 발생시키도록 하는 데 사용됩니다.

객체 타입

TypeScript에서 객체 타입은 객체의 형태를 설명합니다. 객체 프로퍼티의 이름과 타입뿐만 아니라 해당 프로퍼티가 필수인지 선택 사항인지도 지정합니다.

TypeScript에서는 주로 다음 두 가지 방식으로 객체 타입을 정의할 수 있습니다.

인터페이스는 프로퍼티의 이름, 타입, 선택 여부를 지정하여 객체의 형태를 정의합니다.

```
interface User {
    name: string;
    age: number;
    email?: string;
}
```

타입 별칭은 인터페이스와 마찬가지로 객체의 형태를 정의합니다. 하지만 기존 타입이나 기존 타입의 조합을 기반으로 새로운 사용자 지정 타입을 만들 수도 있습니다. 여기에는 유니온 타입, 인터섹션 타입 및 기타 복잡한 타입의 정의가 포함됩니다.

```
type Point = {
    x: number;
    y: number;
};
```

타입을 익명으로 정의할 수도 있습니다.

```
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

튜플 타입(익명)

튜플 타입은 고정된 개수의 요소와 각 요소에 해당하는 타입을 나타내는 배열 타입입니다. 튜플 타입은 특정 개수의 요소와 각 요소의 타입을 고정된 순서로 강제합니다. 튜플 타입은 각 배열 요소의 위치에 특정 의미가 있는, 특정 타입의 값 모음을 나타내고자 할 때 유용합니다.

```
type Point = [number, number];
```

명명된 튜플 타입(레이블 지정)

튜플 타입은 각 요소에 선택적인 레이블이나 이름을 포함할 수 있습니다. 이러한 레이블은 가독성과 도구 지원을 위한 것이며, 튜플로 수행할 수 있는 연산에는 영향을 주지 않습니다.

```
type T = string;
type Tuple1 = [T, T];
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // Named Tuple plus Anonymous Tuple
```

고정 길이 튜플

고정 길이 튜플은 특정 타입을 가진 요소의 개수를 고정하고, 한 번 정의한 후에는 튜플의 길이를 수정하지 못하게 하는 특정 종류의 튜플입니다.

고정 길이 튜플은 특정 개수의 요소와 특정 타입으로 구성된 값 모음을 나타내야 하며, 튜플의 길이와 타입이 실수로 변경되지 않도록 보장하고자 할 때 유용합니다.

```
const x = [10, 'hello'] as const;
x.push(2); // Error
```

유니온 타입

유니온 타입은 여러 타입 중 하나일 수 있는 값을 나타내는 타입입니다. 유니온 타입은 가능한 각 타입 사이에 | 기호를 사용하여 표기합니다.

```
let x: string | number;
x = 'hello'; // Valid
x = 123; // Valid
```

인터섹션 타입

인터섹션 타입은 둘 이상의 타입이 가진 모든 프로퍼티를 포함하는 값을 나타내는 타입입니다. 인터섹션 타입은 각 타입 사이에 & 기호를 사용하여 표기합니다.

```
type X = {
    a: string;
};

type Y = {
    b: string;
};

type J = X & Y; // Intersection

const j: J = {
    a: 'a',
    b: 'b',
};
```

타입 인덱싱

타입 인덱싱은 명시적으로 선언되지 않은 프로퍼티의 타입을 지정하는 인덱스 시그니처를 사용하여 미리 알 수 없는 키로 인덱싱할 수 있는 타입을 정의하는 기능을 의미합니다.

```
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // Returns a
```

값에서 타입 얻기

TypeScript에서 값으로부터 타입 얻기란 타입 추론을 통해 값이나 표현식에서 타입을 자동으로 추론하는 것을 의미합니다.

```
const x = 'x'; // TypeScript infers 'x' as a string literal with 'const' (immutable), but widens it to 'string' with 'let' (reassignable).
```

함수 반환값에서 타입 얻기

함수 반환값으로부터 타입 얻기란 함수의 구현을 바탕으로 반환 타입을 자동으로 추론하는 기능을 의미합니다. 이를 통해 TypeScript는 명시적인 타입 어노테이션 없이 함수가 반환하는 값의 타입을 결정할 수 있습니다.

```
const add = (x: number, y: number) => x + y; // TypeScript can infer that the return type of the function is a number
```

모듈에서 타입 얻기

모듈로부터 타입 얻기란 모듈에서 내보낸 값을 사용하여 해당 타입을 자동으로 추론하는 기능을 의미합니다. 모듈이 특정 타입의 값을 내보내면

TypeScript는 그 정보를 사용하여 해당 값을 다른 모듈로 가져올 때 값의 타입을 자동으로 추론할 수 있습니다.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r is number
```

매핑된 타입

TypeScript의 매핑된 타입을 사용하면 매핑 함수를 통해 각 프로퍼티를 변환하여 기존 타입을 기반으로 새 타입을 만들 수 있습니다. 기존 타입을 매핑하면 같은 정보를 다른 형식으로 나타내는 새 타입을 만들 수 있습니다. 매핑된 타입을 만들려면 `keyof` 연산자를 사용하여 기존 타입의 프로퍼티에 접근한 다음 이를 변경해 새 타입을 생성합니다. 다음 예제에서는:

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
};
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
    foo: ['hello', 'world'],
    bar: [1, 2, 3],
};
```

T의 프로퍼티를 순회하며 각 프로퍼티를 원래 타입의 배열로 만드는 MyMappedType을 정의합니다. 이를 사용해 MyType과 같은 정보를

나타내되 각 프로퍼티가 배열인 MyNewType을 만듭니다.

매핑된 타입 수정자

TypeScript의 매핑된 타입 수정자를 사용하면 기존 타입 내의 프로퍼티를 변환할 수 있습니다.

- `readonly` 또는 `+readonly`: 매핑된 타입의 프로퍼티를 읽기 전용으로 만듭니다.
- `-readonly`: 매핑된 타입의 프로퍼티를 변경할 수 있게 합니다.
- `?`: 매핑된 타입의 프로퍼티를 선택 사항으로 지정합니다.

예제:

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // All properties marked as read-only

type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // All properties marked as mutable

type MyPartial<T> = { [P in keyof T]?: T[P] }; // All properties marked as optional
```

조건부 타입

조건부 타입은 조건에 따라 타입을 만드는 방법으로, 조건의 결과에 따라 생성할 타입이 결정됩니다. `extends` 키워드와 삼항 연산자를 사용하여 두 타입 중 하나를 조건부로 선택하도록 정의합니다.

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;
```

```
type IsMyArrayAnArray = IsArray<typeof myArray>; // Type true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Type false
```

분배 조건부 타입

분배 조건부 타입은 유니온의 각 멤버에 변환을 개별적으로 적용하여 타입을 타입 유니온 전체에 분배할 수 있게 하는 기능입니다. 이는 매핑된 타입이나 고차 타입을 다룰 때 특히 유용할 수 있습니다.

```
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number | boolean | null
```

조건부 타입의 infer 타입 추론

infer 키워드는 조건부 타입에서 해당 타입에 의존하는 제네릭 매개변수의 타입을 추론(추출)하는 데 사용됩니다. 이를 통해 더 유연하고 재사용 가능한 타입 정의를 작성할 수 있습니다.

```
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

미리 정의된 조건부 타입

TypeScript에서 미리 정의된 조건부 타입은 언어에서 제공하는 기본 제공 조건부 타입입니다. 주어진 타입의 특성에 따라 일반적인 타입 변환을 수행하도록 설계되었습니다.

`Exclude<UnionType, ExcludedType>`: Type에서 ExcludedType에 할당할 수 있는 모든 타입을 제거합니다.

`Extract<Type, Union>`: Union에서 Type에 할당할 수 있는 모든 타입을 추출합니다.

`NonNullable<Type>`: Type에서 null과 undefined를 제거합니다.

`ReturnType<Type>`: 함수 Type의 반환 타입을 추출합니다.

`Parameters<Type>`: 함수 Type의 매개변수 타입을 추출합니다.

`Required<Type>`: Type의 모든 프로퍼티를 필수로 만듭니다.

`Partial<Type>`: Type의 모든 프로퍼티를 선택 사항으로 만듭니다.

`Readonly<Type>`: Type의 모든 프로퍼티를 읽기 전용으로 만듭니다.

템플릿 유니온 타입

템플릿 유니온 타입을 사용하면 타입 시스템 안에서 텍스트를 병합하고 조작할 수 있습니다. 예를 들면 다음과 같습니다.

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type  ProductId = `id-${Products}-${Status}`; // "id-p1-active" | "id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

Any 타입

any 타입은 모든 종류의 값(원시 값, 객체, 배열, 함수, 오류, 심볼)을 나타내는 데 사용할 수 있는 특수 타입(범용 슈퍼타입)입니다. 값의 타입을 컴파일 시점에 알 수 없거나 TypeScript 타입 정의가 없는 외부 API 또는 라이브러리의 값을 다룰 때 자주 사용됩니다.

any 타입을 사용하면 값을 아무런 제한 없이 나타내야 한다고 TypeScript 컴파일러에 알리는 것입니다. 코드의 타입 안전성을 최대화하려면 다음 사항을 고려하세요.

- 값의 타입을 정말 알 수 없는 특정 사례로 any 사용을 제한합니다.
- 함수에서 any 타입을 반환하지 마세요. 해당 함수를 사용하는 코드의 타입 안전성이 약화됩니다.
- 컴파일러를 무시해야 한다면 any 대신 @ts-ignore를 사용합니다.

```
let value: any;
value = true; // Valid
value = 7; // Valid
```

Unknown 타입

TypeScript에서 unknown 타입은 타입을 알 수 없는 값을 나타냅니다. 모든 타입의 값을 허용하는 any 타입과 달리, unknown은 특정 방식으로 사용하기 전에 타입 검사 또는 단언이 필요하므로, 더 구체적인 타입으로 먼저 단언하거나 좁히지 않으면 unknown에 어떠한 연산도 수행할 수 없습니다.

unknown 타입은 any와 unknown 자체에만 할당할 수 있으며, any를 대신하는 타입 안전한 선택지입니다.

```
let value: unknown;

let value1: unknown = value; // Valid
let value2: any = value; // Valid
let value3: boolean = value; // Invalid
let value4: number = value; // Invalid

const add = (a: unknown, b: unknown): number | undefined =>
    typeof a === 'number' && typeof b === 'number' ? a + b : undefined;
```

```
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

Void 타입

void 타입은 함수가 값을 반환하지 않음을 나타내는 데 사용됩니다.

```
const sayHello = (): void => {
    console.log('Hello!');
};
```

Never 타입

never 타입은 절대 발생하지 않는 값을 나타냅니다. 절대 반환하지 않거나 오류를 던지는 함수 또는 표현식을 나타내는 데 사용됩니다.

예를 들어 무한 루프는 다음과 같습니다.

```
const infiniteLoop = (): never => {
    while (true) {
        // do something
    }
};
```

오류 던지기:

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

never 타입은 타입 안전성을 보장하고 코드의 잠재적인 오류를 포착하는 데 유용합니다. 다른 타입 및 제어 흐름 문과 함께 사용하면 TypeScript가 더 정밀한 타입을 분석하고 추론하는 데 도움이 됩니다. 예를 들면 다음과 같습니다.

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // move up
            break;
        case 'down':
            // move down
            break;
        default:
            const exhaustiveCheck: never = direction;
            throw new Error(`Unhandled direction: ${exhaustiveCheck}`);
    }
};
```

인터페이스와 타입

공통 구문

TypeScript에서 인터페이스는 객체의 구조를 정의하며 객체가 가져야 하는 프로퍼티 또는 메서드의 이름과 타입을 지정합니다. TypeScript에서 인터페이스를 정의하는 일반적인 구문은 다음과 같습니다.

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
}
```

타입 정의도 이와 유사합니다.

```
type TypeName = {
    property1: Type1;
    // ...
```

```
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};
```

interface InterfaceName 또는 type TypeName: 인터페이스의 이름을 정의합니다. property1: Type1: 인터페이스의 프로퍼티와 해당 타입을 지정합니다. 여러 프로퍼티를 정의할 수 있으며 각 프로퍼티는 세미콜론으로 구분됩니다. method1(arg1: ArgType1, arg2: ArgType2): ReturnType;: 인터페이스의 메서드를 지정합니다. 메서드는 이름, 괄호 안의 매개변수 목록, 반환 타입 순으로 정의됩니다. 여러 메서드를 정의할 수 있으며 각 메서드는 세미콜론으로 구분됩니다.

인터페이스 예제:

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

타입 예제:

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

TypeScript에서 타입은 데이터의 형태를 정의하고 타입 검사를 강제하는 데 사용됩니다. TypeScript에서 타입을 정의하는 일반적인 구문은 구체적인 사용 사례에 따라 여러 가지가 있습니다. 다음은 몇 가지 예제입니다.

기본 타입

```
let myNumber: number = 123; // number type
let myBoolean: boolean = true; // boolean type
let myArray: string[] = ['a', 'b']; // array of strings
let myTuple: [string, number] = ['a', 123]; // tuple
```

객체와 인터페이스

```
const x: { name: string; age: number } = { name: 'Simon', age: 7 };
```

유니온 타입과 인터섹션 타입

```
type MyType = string | number; // Union type
let myUnion: MyType = 'hello'; // Can be a string
myUnion = 123; // Or a number

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Intersection type
let myCombined: CombinedType = { name: 'John', age: 25 }; // Object with
both name and age properties
```

내장 원시 타입

TypeScript에는 변수, 함수 매개변수, 반환 타입을 정의하는 데 사용할 수 있는 여러 기본 제공 원시 타입이 있습니다.

- `number`: 정수와 부동 소수점 수를 포함한 숫자 값을 나타냅니다.
- `string`: 텍스트 데이터를 나타냅니다.
- `boolean`: true 또는 false일 수 있는 논리 값을 나타냅니다.
- `null`: 값이 없음을 나타냅니다.
- `undefined`: 할당되지 않았거나 정의되지 않은 값을 나타냅니다.
- `symbol`: 고유 식별자를 나타냅니다. 심볼은 일반적으로 객체 프로퍼티의 키로 사용됩니다.

- `bigint`: 임의 정밀도 정수를 나타냅니다.
- `any`: 동적 타입 또는 알 수 없는 타입을 나타냅니다. any 타입 변수는 모든 타입의 값을 담을 수 있으며 타입 검사를 우회합니다.
- `void`: 어떠한 타입도 없음을 나타냅니다. 값을 반환하지 않는 함수의 반환 타입으로 흔히 사용됩니다.
- `never`: 절대 발생하지 않는 값의 타입을 나타냅니다. 일반적으로 오류를 던지거나 무한 루프에 들어가는 함수의 반환 타입으로 사용됩니다.

일반적인 내장 JS 객체

TypeScript는 JavaScript의 상위 집합이며 일반적으로 사용되는 모든 기본 제공 JavaScript 객체를 포함합니다. Mozilla Developer Network(MDN) 문서 웹사이트에서 이러한 객체의 전체 목록을 확인할 수 있습니다. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

일반적으로 사용되는 기본 제공 JavaScript 객체 중 일부는 다음과 같습니다.

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String
- RegExp

- Array
- Map
- Set
- Promise
- Intl

오버로드

TypeScript의 함수 오버로드를 사용하면 하나의 함수 이름에 여러 함수 시그니처를 정의할 수 있어, 여러 방식으로 호출할 수 있는 함수를 정의할 수 있습니다. 다음은 예제입니다.

```
// Overloads
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// Implementation
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // Valid
sayHi(['aa', 'bb']); // Valid
```

다음은 class 안에서 함수 오버로드를 사용하는 또 다른 예제입니다.

```
class Greeter {
    message: string;
```

```
    constructor(message: string) {
        this.message = message;
    }

    // overload
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;

    // implementation
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
            return name.map(name => `${this.message}, ${name}!`);
        }
        throw new Error('value is invalid');
    }
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

병합과 확장

병합과 확장은 타입 및 인터페이스 작업과 관련된 서로 다른 두 개념을 가리킵니다.

병합을 사용하면 이름이 같은 여러 선언을 하나의 정의로 결합할 수 있습니다. 예를 들어 같은 이름의 인터페이스를 여러 번 정의하는 경우입니다.

```
interface X {
    a: string;
}

interface X {
```

```
    b: number;
}

const person: X = {
    a: 'a',
    b: 7,
};
```

확장은 기존 타입 또는 인터페이스를 확장하거나 상속하여 새 타입을 만드는 기능을 가리킵니다. 원래 정의를 수정하지 않고 기존 타입에 프로퍼티나 메서드를 추가하는 메커니즘입니다. 예제:

```
interface Animal {
    name: string;
    eat(): void;
}

interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

타입과 인터페이스의 차이점

선언 병합(보강):

인터페이스는 선언 병합을 지원합니다. 즉, 같은 이름으로 여러 인터페이스를 정의하면 TypeScript가 결합된 프로퍼티와 메서드를 가진 하나의 인터페이스로 병합합니다. 반면 타입은 선언 병합을 지원하지 않습니다. 이는 원래 정의를 수정하거나 누락되었거나 잘못된 타입을 패치하지 않고 추가 기능을 더하거나 기존 타입을 사용자 지정하려 할 때 유용할 수 있습니다.

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
    y: 'yy',
};
```

다른 타입/인터페이스 확장:

타입과 인터페이스 모두 다른 타입/인터페이스를 확장할 수 있지만 구문은 다릅니다. 인터페이스에서는 `extends` 키워드를 사용하여 다른 인터페이스의 프로퍼티와 메서드를 상속합니다. 그러나 인터페이스는 유니온 타입과 같은 복합 타입을 확장할 수 없습니다.

```
interface A {
    x: string;
    y: number;
}
interface B extends A {
    z: string;
}
const car: B = {
    x: 'x',
```

```
    y: 123,
    z: 'z',
};
```

타입에서는 & 연산자를 사용하여 여러 타입을 하나의 타입(인터섹션)으로 결합합니다.

```
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};

const c: B = {
    x: 'x',
    y: 123,
    j: 'j',
};
```

유니온 타입과 인터섹션 타입:

유니온 타입과 인터섹션 타입을 정의할 때는 타입 별칭이 더 유연합니다. type 키워드를 사용하면 | 연산자로 유니온 타입을, & 연산자로 인터섹션 타입을 쉽게 만들 수 있습니다. 인터페이스도 유니온 타입을 간접적으로 나타낼 수 있지만 인터섹션 타입을 기본적으로 지원하지 않습니다.

```
type Department = 'dep-x' | 'dep-y'; // Union

type Person = {
    name: string;
    age: number;
```

```
};

type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // Intersection
```

인터페이스를 사용한 예제:

```
interface A {
    x: 'x';
}
interface B {
    y: 'y';
}

type C = A | B; // Union of interfaces
```

클래스

클래스 공통 구문

TypeScript에서 class 키워드는 클래스를 정의하는 데 사용됩니다. 아래에서 예제를 확인할 수 있습니다.

```
class Person {
    private name: string;
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public sayHi(): void {
```

```
        console.log(
            `Hello, my name is ${this.name} and I am ${this.age} years old.`
        );
    }
}
```

class 키워드는 "Person"이라는 클래스를 정의하는 데 사용됩니다.

이 클래스에는 두 개의 private 프로퍼티가 있습니다. string 타입의 name과 number 타입의 age입니다.

생성자는 constructor 키워드를 사용하여 정의됩니다. name과 age를 매개변수로 받아 해당 프로퍼티에 할당합니다.

클래스에는 인사 메시지를 기록하는 sayHi라는 public 메서드가 있습니다.

TypeScript에서 클래스의 인스턴스를 만들려면 new 키워드 다음에 클래스 이름과 괄호 ()를 사용할 수 있습니다. 예를 들면 다음과 같습니다.

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Output: Hello, my name is John Doe and I am 25 years old.
```

생성자

생성자는 클래스의 인스턴스가 생성될 때 객체의 프로퍼티를 초기화하는 데 사용되는 클래스 내의 특수 메서드입니다.

```
class Person {
    public name: string;
    public age: number;

    constructor(name: string, age: number) {
        this.name = name;
```

```
        this.age = age;
    }

    sayHello() {
        console.log(
            `Hello, my name is ${this.name} and I'm ${this.age} years old.`
        );
    }
}

const john = new Person('Simon', 17);
john.sayHello();
```

다음 구문을 사용하여 생성자를 오버로드할 수 있습니다.

```
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

TypeScript에서는 여러 생성자 오버로드를 정의할 수 있지만 모든 오버로드와 호환되어야 하는 구현은 하나만 둘 수 있습니다. 이는 선택적 매

개변수를 사용하여 구현할 수 있습니다.

```
class Person {
    name: string;
    age: number;

    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }

    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

Private 및 Protected 생성자

TypeScript에서는 생성자를 private 또는 protected로 표시하여 접근성과 사용을 제한할 수 있습니다.

private 생성자: 클래스 자체 내에서만 호출할 수 있습니다. private 생성자는 싱글턴 패턴을 강제하거나 인스턴스 생성을 클래스 내의 팩토리

메서드로 제한하려는 경우에 자주 사용됩니다.

protected 생성자: 직접 인스턴스화해서는 안 되지만 서브클래스가 확장할 수 있는 기본 클래스를 만들려는 경우에 유용합니다.

```
class BaseClass {
    protected constructor() {}
}

class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }
}

// Attempting to instantiate the base class directly will result in an error
// const baseObj = new BaseClass(); // Error: Constructor of class 'BaseClass' is protected.

// Create an instance of the derived class
const derivedObj = new DerivedClass(10);
```

접근 제한자

접근 제한자 private, protected, public은 TypeScript 클래스에서 프로퍼티와 메서드 같은 클래스 멤버의 가시성과 접근성을 제어하는 데 사용됩니다. 이러한 제한자는 캡슐화를 강제하고 클래스 내부 상태에 접근하고 이를 수정하기 위한 경계를 설정하는 데 필수적입니다.

private 제한자는 클래스 멤버에 대한 접근을 해당 멤버를 포함하는 클래스 내부로만 제한합니다.

protected 제한자는 해당 멤버를 포함하는 클래스와 파생 클래스 내부에서 클래스 멤버에 접근할 수 있게 합니다.

public 제한자는 클래스 멤버에 제한 없이 접근할 수 있게 하여 어디서든 접근할 수 있도록 합니다.

Get과 Set

getter와 setter는 클래스 프로퍼티에 대한 사용자 지정 접근 및 수정 동작을 정의할 수 있게 하는 특수 메서드입니다. 객체의 내부 상태를 캡슐화하고 프로퍼티 값을 가져오거나 설정할 때 추가 로직을 제공할 수 있습니다. TypeScript에서 getter와 setter는 각각 get과 set 키워드를 사용하여 정의됩니다. 다음은 예제입니다.

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
    }
}
```

클래스의 자동 접근자

TypeScript 버전 4.9에서는 향후 제공될 ECMAScript 기능인 자동 접근자를 지원합니다. 자동 접근자는 클래스 프로퍼티와 유사하지만 “accessor” 키워드로 선언됩니다.

```
class Animal {
    accessor name: string;

    constructor(name: string) {
        this.name = name;
    }
}
```

자동 접근자는 접근할 수 없는 프로퍼티에 대해 작동하는 private get 및 set 접근자로 "디슈거링"됩니다.

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
    }
}
```

this

TypeScript에서 this 키워드는 메서드 또는 생성자 내에서 클래스의 현재 인스턴스를 가리킵니다. 클래스 자체의 범위 안에서 클래스의 프로퍼티와 메서드에 접근하고 이를 수정할 수 있게 합니다. 객체 자체의 메서드 안에서 객체의 내부 상태에 접근하고 이를 조작하는 방법을 제공합니다.

```
class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
    }
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}

const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

매개변수 프로퍼티

매개변수 프로퍼티를 사용하면 생성자 매개변수에서 클래스 프로퍼티를 직접 선언하고 초기화하여 반복적인 코드를 피할 수 있습니다. 예를 들면 다음과 같습니다.

```
class Person {
    constructor(
        private name: string,
        public age: number
    ) {
        // The "private" and "public" keywords in the constructor
          // automatically declare and initialize the corresponding class
properties.
    }
    public introduce(): void {
        console.log(
            `Hello, my name is ${this.name} and I am ${this.age} years old.`
        );
    }
}
```

```
const person = new Person('Alice', 25);
person.introduce();
```

추상 클래스

추상 클래스는 TypeScript에서 주로 상속에 사용됩니다. 서브클래스가 상속할 수 있는 공통 프로퍼티와 메서드를 정의하는 방법을 제공합니다. 공통 동작을 정의하고 서브클래스가 특정 메서드를 구현하도록 강제하려는 경우 유용합니다. 추상 기본 클래스는 서브클래스에 공유 인터페이스와 공통 기능을 제공하는 클래스 계층 구조를 만들 때 유용합니다.

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // Output: Whiskers meows.
```

제네릭과 함께 사용하기

제네릭 클래스는 여러 타입을 대상으로 작동할 수 있는 재사용 가능한 클래스를 정의할 수 있게 합니다.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }

    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

데코레이터

데코레이터는 메타데이터를 추가하고 동작을 수정하거나 검증하며 대상 요소의 기능을 확장하는 메커니즘을 제공합니다. 데코레이터는 런타임에 실행되는 함수입니다. 하나의 선언에 여러 데코레이터를 적용할 수 있습니다.

데코레이터는 실험적 기능이며 다음 예제는 ES6를 사용하는 TypeScript 버전 5 이상에서만 호환됩니다.

버전 5 이전 TypeScript에서는 `experimentalDecorators` 프로퍼티를 `tsconfig.json`에서 사용하거나 명령줄에서 `--experimentalDecorators`를 사용하여 활성화해야 합니다(하지만 다음 예제는 작동하지 않습니다).

데코레이터의 일반적인 사용 사례는 다음과 같습니다.

- 프로퍼티 변경 감시.
- 메서드 호출 감시.
- 추가 프로퍼티 또는 메서드 추가.
- 런타임 검증.
- 자동 직렬화 및 역직렬화.
- 로깅.
- 권한 부여 및 인증.
- 오류 방지.

참고: 버전 5의 데코레이터는 매개변수 데코레이팅을 허용하지 않습니다.

데코레이터의 종류:

클래스 데코레이터

클래스 데코레이터는 프로퍼티나 메서드를 추가하거나 클래스의 인스턴스를 수집하는 등 기존 클래스를 확장하는 데 유용합니다. 다음 예제에서는 클래스를 문자열 표현으로 변환하는 `toString` 메서드를 추가합니다.

```
type Constructor<T = {}> = new (...args: any[]) => T;

function toString<Class extends Constructor>(
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
```

```
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
        }
    };
}

@toString
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
const person = new Person('Simon');
/* Logs:
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/
```

프로퍼티 데코레이터

프로퍼티 데코레이터는 초기화 값을 변경하는 등 프로퍼티의 동작을 수정하는 데 유용합니다. 다음 코드에는 프로퍼티가 항상 대문자가 되도록 설정하는 스크립트가 있습니다.

```
function upperCase<T>(
    target: undefined,
```

```
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}

class MyClass {
    @upperCase
    prop1 = 'hello!';
}

console.log(new MyClass().prop1); // Logs: HELLO!
```

메서드 데코레이터

메서드 데코레이터를 사용하면 메서드의 동작을 변경하거나 향상할 수 있습니다. 아래는 간단한 로거의 예제입니다.

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

    function replacementMethod(this: This, ...args: Args): Return {
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
    }
```

```
    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
    }
}

new MyClass().sayHello();
```

다음과 같이 출력됩니다.

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

Getter 및 Setter 데코레이터

getter 및 setter 데코레이터를 사용하면 클래스 접근자의 동작을 변경하거나 향상할 수 있습니다. 예를 들어 프로퍼티 할당을 검증하는 데 유용합니다. 다음은 getter 데코레이터의 간단한 예제입니다.

```
function range<This, Return extends number>(min: number, max: number) {
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
                throw 'Invalid';
            }
            Object.defineProperty(this, context.name, {
```

```
                value,
                enumerable: true,
            });
            return value;
        };
    };
}

class MyClass {
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // Valid: 10

const obj2 = new MyClass(999);
console.log(obj2.getValue); // Throw: Invalid!
```

데코레이터 메타데이터

데코레이터 메타데이터는 데코레이터가 모든 클래스에서 메타데이터를 적용하고 활용하는 과정을 간소화합니다. 데코레이터는 컨텍스트 객체의 새로운 metadata 프로퍼티에 접근할 수 있으며, 이 프로퍼티는 원시 값과 객체 모두의 키 역할을 할 수 있습니다. 메타데이터 정보는 `Symbol.metadata`를 통해 클래스에서 접근할 수 있습니다.

메타데이터는 디버깅, 직렬화 또는 데코레이터를 사용한 의존성 주입 등 다양한 목적으로 사용할 수 있습니다.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Simple polyfill

type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
     | ClassMethodDecoratorContext; // Context contains property metadata:
DecoratorMetadata

function setMetadata(_target: any, context: Context) {
    // Set the metadata object with a primitive value
    context.metadata[context.name] = true;
}

class MyClass {
    @setMetadata
    a = 123;

    @setMetadata
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Get metadata information

console.log(JSON.stringify(metadata)); // {"bar":true,"baz":true,"foo":true}
```

상속

상속은 클래스가 기본 클래스 또는 슈퍼클래스라고 하는 다른 클래스의 프로퍼티와 메서드를 상속할 수 있는 메커니즘을 가리킵니다. 자식 클래스 또는 서브클래스라고도 하는 파생 클래스는 새로운 프로퍼티와 메서드를 추가하거나 기존 항목을 재정의하여 기본 클래스의 기능을 확장하고 특수화할 수 있습니다.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// Create an instance of the base class
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound
```

```
// Create an instance of the derived class
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

TypeScript는 전통적인 의미의 다중 상속을 지원하지 않으며 대신 하나의 기본 클래스에서 상속할 수 있습니다. TypeScript는 다중 인터페이스를 지원합니다. 인터페이스는 객체 구조에 대한 계약을 정의할 수 있으며 클래스는 여러 인터페이스를 구현할 수 있습니다. 이를 통해 클래스가 여러 소스로부터 동작과 구조를 상속할 수 있습니다.

```
interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Flying...');
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

JavaScript와 마찬가지로 TypeScript의 class 키워드는 흔히 문법적 설탕이라고 합니다. 클래스 기반 방식으로 객체를 만들고 다루는 데 더 익

숙한 구문을 제공하기 위해 ECMAScript 2015(ES6)에 도입되었습니다. 그러나 TypeScript는 JavaScript의 상위 집합이므로 궁극적으로 JavaScript로 컴파일되며, JavaScript의 핵심은 여전히 프로토타입 기반이라는 점에 유의해야 합니다.

정적 멤버

TypeScript에는 정적 멤버가 있습니다. 클래스의 정적 멤버에 접근하려면 객체를 생성할 필요 없이 클래스 이름 다음에 점을 사용할 수 있습니다.

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

프로퍼티 초기화

TypeScript에서 클래스의 프로퍼티를 초기화하는 방법은 여러 가지가 있습니다.

인라인:

다음 예제에서는 클래스의 인스턴스가 생성될 때 이러한 초기값이 사용됩니다.

```
class MyClass {
    property1: string = 'default value';
    property2: number = 42;
}
```

생성자에서:

```
class MyClass {
    property1: string;
    property2: number;

    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

생성자 매개변수 사용:

```
class MyClass {
    constructor(
        private property1: string = 'default value',
        public property2: number = 42
    ) {
            // There is no need to assign the values to the properties
explicitly.
    }
    log() {
        console.log(this.property2);
    }
}
const x = new MyClass();
x.log();
```

메서드 오버로딩

메서드 오버로드를 사용하면 클래스가 이름은 같지만 매개변수 타입이나 매개변수 수가 다른 여러 메서드를 가질 수 있습니다. 이를 통해 전달된 인수에 따라 메서드를 여러 방식으로 호출할 수 있습니다.

```
class MyClass {
    add(a: number, b: number): number; // Overload signature 1
    add(a: string, b: string): string; // Overload signature 2

    add(a: number | string, b: number | string): number | string {
        if (typeof a === 'number' && typeof b === 'number') {
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
}

const r = new MyClass();
console.log(r.add(10, 5)); // Logs 15
```

제네릭

제네릭을 사용하면 여러 타입과 함께 동작할 수 있는 재사용 가능한 컴포넌트와 함수를 만들 수 있습니다. 제네릭을 사용하면 타입, 함수, 인터페이스를 매개변수화하여 사전에 타입을 명시적으로 지정하지 않고도 여러 타입을 대상으로 동작하게 할 수 있습니다.

제네릭을 사용하면 코드를 더 유연하고 재사용 가능하게 만들 수 있습니다.

제네릭 타입

제네릭 타입을 정의하려면 다음과 같이 꺾쇠괄호(<>)를 사용하여 타입 매개변수를 지정합니다.

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);

const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

제네릭 클래스

제네릭은 클래스에도 적용할 수 있으며, 이를 통해 클래스가 타입 매개변수를 사용하여 여러 타입과 함께 동작할 수 있습니다. 이는 타입 안전성을 유지하면서 여러 데이터 타입을 대상으로 동작할 수 있는 재사용 가능한 클래스 정의를 만드는 데 유용합니다.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123
```

```
const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

제네릭 제약 조건

제네릭 매개변수는 extends 키워드 뒤에 타입 매개변수가 충족해야 하는 타입이나 인터페이스를 지정하여 제약할 수 있습니다.

다음 예제에서 T가 유효하려면 올바르게 타입이 지정된 length 프로퍼티가 있어야 합니다.

```
const printLen = <T extends { length: number }>(value: T): void => {
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
printLen({ length: 10 }); // 10
printLen(123); // Invalid
```

버전 3.4 RC에서 도입된 주목할 만한 제네릭 기능으로 제네릭 타입 인수를 전파하는 고차 함수 타입 추론이 있습니다.

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

이 기능을 사용하면 함수형 프로그래밍에서 흔히 사용하는 타입 안전한 포인트프리 스타일 프로그래밍이 더 쉬워집니다.

제네릭의 문맥적 타입 좁히기

제네릭의 문맥적 타입 좁히기는 컴파일러가 제네릭 매개변수가 사용되는 문맥을 기반으로 해당 매개변수의 타입을 좁힐 수 있게 하는 TypeScript의 메커니즘입니다. 조건문에서 제네릭 타입을 다룰 때 유용합니다.

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // Value is narrowed down to type 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // Value is narrowed down to type 'number'
        console.log(value.toFixed(2));
    }
}

process('hello'); // 5
process(3.14159); // 3.14
```

소거된 구조적 타입

TypeScript에서 객체가 특정 타입과 정확히 일치할 필요는 없습니다. 예를 들어 인터페이스의 요구 사항을 충족하는 객체를 만들면, 객체와 해당 인터페이스 사이에 명시적인 연결이 없더라도 그 인터페이스가 필요한 곳에서 해당 객체를 사용할 수 있습니다. 예제:

```
type NameProp1 = {
    prop1: string;
};
```

```
function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
    prop1: 'Origin',
};

log(obj); // Valid
```

네임스페이스 지정

TypeScript에서 네임스페이스는 코드를 논리적 컨테이너로 구성하는 데 사용되며, 이름 충돌을 방지하고 관련 코드를 함께 그룹화할 수 있게 합니다. export 키워드를 사용하면 모듈 외부에서 네임스페이스에 접근할 수 있습니다.

```
export namespace MyNamespace {
    export interface MyInterface1 {
        prop1: boolean;
    }
    export interface MyInterface2 {
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

심볼

심볼은 프로그램이 실행되는 동안 전역적으로 고유함이 보장되는 불변 값을 나타내는 원시 데이터 타입입니다.

심볼은 객체 프로퍼티의 키로 사용할 수 있으며 열거할 수 없는 프로퍼티를 만드는 방법을 제공합니다.

```
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

이제 WeakMap과 WeakSet에서 심볼을 키로 사용할 수 있습니다.

트리플 슬래시 지시문

트리플 슬래시 지시문은 컴파일러에 파일 처리 방법을 지시하는 특별한 주석입니다. 이러한 지시문은 연속된 세 개의 슬래시(///)로 시작하며 일반적으로 TypeScript 파일의 맨 위에 배치되고 런타임 동작에는 영향을 주지 않습니다.

트리플 슬래시 지시문은 외부 의존성을 참조하고, 모듈 로딩 동작을 지정하며, 특정 컴파일러 기능을 활성화하거나 비활성화하는 등의 용도로 사용됩니다. 몇 가지 예는 다음과 같습니다.

선언 파일 참조:

```
/// <reference path="path/to/declaration/file.d.ts" />
```

모듈 형식 지정:

```
/// <amd|commonjs|system|umd|es6|es2015|none>
```

컴파일러 옵션 활성화. 다음 예제에서는 엄격 모드를 활성화합니다.

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

타입 조작

타입에서 타입 생성하기

기존 타입을 조합하거나 조작하거나 변환하여 새로운 타입을 만들 수 있습니다.

인터섹션 타입(&):

여러 타입을 하나의 타입으로 결합할 수 있습니다.

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // Intersection of A and B
const obj: C = { foo: 42, bar: 'hello' };
```

유니온 타입(|):

여러 타입 중 하나가 될 수 있는 타입을 정의할 수 있습니다.

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

매핑된 타입:

기존 타입의 프로퍼티를 변환하여 새로운 타입을 만들 수 있습니다.

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // Properties become read-only
```

조건부 타입:

몇 가지 조건을 기반으로 타입을 만들 수 있습니다.

```
type ExtractParam<T> = T extends (param: infer P) => any ? P : never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

인덱스 접근 타입

TypeScript에서는 인덱스 Type[Key]를 사용하여 다른 타입에 있는 프로퍼티의 타입에 접근하고 이를 조작할 수 있습니다.

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

유틸리티 타입

타입을 조작하는 데 사용할 수 있는 여러 내장 유틸리티 타입이 있습니다. 다음은 가장 일반적으로 사용되는 타입의 목록입니다.

Awaited<T>

Promise 타입을 재귀적으로 언래핑하는 타입을 생성합니다.

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

T의 모든 프로퍼티를 선택 사항으로 설정한 타입을 생성합니다.

```
type Person = {
    name: string;
    age: number;
};

type A = Partial<Person>; // { name?: string | undefined; age?: number | undefined; }
```

Required<T>

T의 모든 프로퍼티를 필수로 설정한 타입을 생성합니다.

```
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

T의 모든 프로퍼티를 읽기 전용으로 설정한 타입을 생성합니다.

```
type Person = {
    name: string;
    age: number;
};

type A = Readonly<Person>;

const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // Invalid
```

Record<K, T>

타입이 T인 K 프로퍼티 집합을 가진 타입을 생성합니다.

```
type Product = {
    name: string;
    price: number;
};

const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

Pick<T, K>

T에서 지정된 프로퍼티 K를 선택하여 타입을 생성합니다.

```
type Product = {
    name: string;
```

```
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

Omit<T, K>

T에서 지정된 프로퍼티 K를 생략하여 타입을 생성합니다.

```
type Product = {
    name: string;
    price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }
```

Exclude<T, U>

T에서 U 타입의 모든 값을 제외하여 타입을 생성합니다.

```
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

Extract<T, U>

T에서 U 타입의 모든 값을 추출하여 타입을 생성합니다.

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

NonNullable<T>

T에서 null과 undefined를 제외하여 타입을 생성합니다.

```ts
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

함수 타입 T의 매개변수 타입을 추출합니다.

```ts
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

생성자 함수 타입 T의 매개변수 타입을 추출합니다.

```ts
class Person {
    constructor(
        public name: string,
        public age: number
    ) {}
}
type PersonConstructorParams = ConstructorParameters<typeof Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

ReturnType<T>

함수 타입 T의 반환 타입을 추출합니다.

```ts
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

InstanceType<T>

클래스 타입 T의 인스턴스 타입을 추출합니다.

```
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    sayHello() {
        console.log(`Hello, my name is ${this.name}!`);
    }
}

type PersonInstance = InstanceType<typeof Person>;

const person: PersonInstance = new Person('John');

person.sayHello(); // Hello, my name is John!
```

ThisParameterType<T>

함수 타입 T에서 'this' 매개변수의 타입을 추출합니다.

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type PersonThisType = ThisParameterType<Person['greet']>; // Person
```

OmitThisParameter<T>

함수 타입 T에서 'this' 매개변수를 제거합니다.

```ts
function capitalize(this: String) {
    return this[0].toUpperCase() + this.substring(1).toLowerCase();
}

type CapitalizeType = OmitThisParameter<typeof capitalize>; // () => string
```

ThisType<T>

문맥적 this 타입을 나타내는 마커 역할을 합니다.

```ts
type Logger = {
    log: (error: string) => void;
};

let helperFunctions: { [name: string]: Function } & ThisType<Logger> = {
    hello: function () {
        this.log('some error'); // Valid as "log" is a part of "this".
        this.update(); // Invalid
    },
};
```

Uppercase<T>

입력 타입 T의 이름을 대문자로 변환합니다.

```ts
type MyType = Uppercase<'abc'>; // "ABC"
```

Lowercase<T>

입력 타입 T의 이름을 소문자로 변환합니다.

```ts
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

입력 타입 T 이름의 첫 글자를 대문자로 변환합니다.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

입력 타입 T 이름의 첫 글자를 소문자로 변환합니다.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer는 제네릭 함수 범위 내에서 타입이 자동으로 추론되는 것을 차단하도록 설계된 유틸리티 타입입니다.

예제:

```
// Automatic inference of types within the scope of a generic function.
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // Type here is ("a" | "b" | "c")[]
```

NoInfer를 사용한 경우:

```
// Example function that uses NoInfer to prevent type inference
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // Error: Type Argument of type '"c"' is
not assignable to parameter of type '"a" | "b"'.
```

기타

오류 및 예외 처리

TypeScript에서는 표준 JavaScript 오류 처리 메커니즘을 사용하여 오류를 포착하고 처리할 수 있습니다.

Try-Catch-Finally 블록:

```
try {
    // Code that might throw an error
} catch (error) {
    // Handle the error
} finally {
    // Code that always executes, finally is optional
}
```

여러 유형의 오류를 처리할 수도 있습니다.

```
try {
    // Code that might throw different types of errors
} catch (error) {
    if (error instanceof TypeError) {
        // Handle TypeError
    } else if (error instanceof RangeError) {
        // Handle RangeError
    } else {
        // Handle other errors
    }
}
```

사용자 정의 오류 타입:

Error `class`를 확장하여 더 구체적인 오류를 지정할 수 있습니다.

```
class CustomError extends Error {
    constructor(message: string) {
```

```
        super(message);
        this.name = 'CustomError';
    }
}

throw new CustomError('This is a custom error.');
```

믹스인 클래스

믹스인 클래스를 사용하면 여러 클래스의 동작을 하나의 클래스로 결합하고 구성할 수 있습니다. 깊은 상속 체인 없이 기능을 재사용하고 확장할 수 있습니다.

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
    deselect() {
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// Extend MyClass to include the behavior of Identifiable and Selectable
```

```
interface MyClass extends Identifiable, Selectable {}

// Function to apply mixins to a class
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {
        Object.getOwnPropertyNames(baseCtor.prototype).forEach(name => {
            let descriptor = Object.getOwnPropertyDescriptor(
                baseCtor.prototype,
                name
            );
            if (descriptor) {
                Object.defineProperty(source.prototype, name, descriptor);
            }
        });
    });
}

// Apply the mixins to MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();
```

비동기 언어 기능

TypeScript는 JavaScript의 상위 집합이므로 다음과 같은 JavaScript의 비동기 언어 기능이 내장되어 있습니다.

Promise:

Promise는 `.then()` 및 `.catch()`와 같은 메서드를 사용하여 성공 및 오류 조건을 처리함으로써 비동기 작업과 그 결과를 다루는 방법입니다.

더 알아보기: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

Async/await 키워드는 Promise를 다룰 때 더 동기적으로 보이는 구문을 제공하는 방법입니다. `async` 키워드는 비동기 함수를 정의하는 데 사용되며, `await` 키워드는 비동기 함수 내에서 Promise가 이행되거나 거부될 때까지 실행을 일시 중지하는 데 사용됩니다.

더 알아보기: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

다음 API는 TypeScript에서 잘 지원됩니다.

Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Worker: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Worker: https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

이터레이터와 제너레이터

이터레이터와 제너레이터 모두 TypeScript에서 잘 지원됩니다.

이터레이터는 이터레이터 프로토콜을 구현하는 객체로, 컬렉션이나 시퀀스의 요소에 하나씩 접근하는 방법을 제공합니다. 이터레이션의 다음 요소를 가리키는 포인터가 포함된 구조입니다. 이터레이터에는 next() 메서드가 있으며, 이 메서드는 시퀀스의 다음 값과 시퀀스가 done 상태인지 나타내는 불리언 값을 함께 반환합니다.

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
    }

    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
```

```
    console.log(num);
}
```

제너레이터는 function* 구문을 사용하여 정의하는 특수 함수로, 이터레이터 생성을 간소화합니다. yield 키워드로 값의 시퀀스를 정의하며 값이 요청될 때 실행을 자동으로 일시 중지하고 다시 시작합니다.

제너레이터를 사용하면 이터레이터를 더 쉽게 만들 수 있으며, 특히 크거나 무한한 시퀀스를 다룰 때 유용합니다.

예제:

```
function* numberGenerator(start: number, end: number): Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

TypeScript는 비동기 이터레이터와 비동기 제너레이터도 지원합니다.

더 알아보기:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

TsDocs JSDoc 참조

JavaScript 코드베이스로 작업할 때 추가 어노테이션이 포함된 JSDoc 주석으로 타입 정보를 제공하여 TypeScript가 올바른 타입을 추론하도록 도울 수 있습니다.

예제:

```
/**
 * Computes the power of a given number
 * @constructor
 * @param {number} base - The base value of the expression
 * @param {number} exponent - The exponent value of the expression
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent: number): number
```

전체 문서는 다음 링크에서 제공됩니다. https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

버전 3.7부터 JavaScript JSDoc 구문으로 .d.ts 타입 정의를 생성할 수 있습니다. 자세한 내용은 다음에서 확인할 수 있습니다. https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

@types

@types 조직 아래의 패키지는 기존 JavaScript 라이브러리나 모듈의 타입 정의를 제공하기 위한 특별한 패키지 이름 지정 규칙을 따릅니다. 예를 들어 다음을 사용하면:

```
npm install --save-dev @types/lodash
```

현재 프로젝트에 lodash의 타입 정의가 설치됩니다.

@types 패키지의 타입 정의에 기여하려면 https://github.com/DefinitelyTyped/DefinitelyTyped에 풀 리퀘스트를 제출하세요.

JSX

JSX(JavaScript XML)는 JavaScript 또는 TypeScript 파일 내에서 HTML과 유사한 코드를 작성할 수 있게 하는 JavaScript 언어 구문의 확장입니다. 일반적으로 React에서 HTML 구조를 정의하는 데 사용됩니다.

TypeScript는 타입 검사와 정적 분석을 제공하여 JSX의 기능을 확장합니다.

JSX를 사용하려면 jsx 컴파일러 옵션을 tsconfig.json 파일에서 설정해야 합니다. 일반적인 두 가지 구성 옵션은 다음과 같습니다.

- "preserve": JSX를 변경하지 않은 채 .jsx 파일을 내보냅니다. 이 옵션은 TypeScript가 JSX 구문을 그대로 유지하고 컴파일 과정에서 변환하지 않도록 합니다. Babel처럼 변환을 처리하는 별도의 도구가 있는 경우 이 옵션을 사용할 수 있습니다.
- "react": TypeScript에 내장된 JSX 변환을 활성화합니다. React.createElement가 사용됩니다.

모든 옵션은 다음에서 확인할 수 있습니다. https://www.typescriptlang.org/tsconfig#jsx

ES6 모듈

TypeScript는 ES6(ECMAScript 2015)와 그 이후의 여러 버전을 지원합니다. 즉, 화살표 함수, 템플릿 리터럴, 클래스, 모듈, 구조 분해 등과 같은 ES6 구문을 사용할 수 있습니다.

프로젝트에서 ES6 기능을 활성화하려면 tsconfig.json에서 `target` 프로퍼티를 지정할 수 있습니다.

구성 예제:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

ES7 지수 연산자

지수(**) 연산자는 첫 번째 피연산자를 두 번째 피연산자의 거듭제곱으로 올려 얻은 값을 계산합니다. `Math.pow()`와 유사하게 작동하지만 BigInt도 피연산자로 받을 수 있습니다. TypeScript는 tsconfig.json 파일의 `target`을 `es2016` 이상으로 설정하여 이 연산자를 완전히 지원합니다.

```
console.log(2 ** (2 ** 2)); // 16
```

for-await-of 문

이 기능은 TypeScript에서 완전히 지원되는 JavaScript 기능으로, 대상 버전이 es2018일 때 비동기 이터러블 객체를 순회할 수 있게 합니다.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
    for await (const num of asyncNumbers()) {
        console.log(num);
    }
})();
```

new.target 메타 프로퍼티

TypeScript에서는 new.target 메타 프로퍼티를 사용하여 함수나 생성자가 new 연산자로 호출되었는지 확인할 수 있습니다. 이를 통해 생성자 호출의 결과로 객체가 생성되었는지 감지할 수 있습니다.

```
class Parent {
    constructor() {
        console.log(new.target); // Logs the constructor function used to
create an instance
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}
```

```
const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

동적 import 표현식

TypeScript가 지원하는 동적 import에 관한 ECMAScript 제안을 사용하면 조건에 따라 모듈을 로드하거나 필요할 때 지연 로드할 수 있습니다.

TypeScript의 동적 import 표현식 구문은 다음과 같습니다.

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
        const widget = await import('./widget'); // Dynamic import
        widget.render(container);
    }
}

renderWidget();
```

“tsc -watch”

이 명령은 `--watch` 매개변수로 TypeScript 컴파일러를 시작하여 TypeScript 파일이 수정될 때마다 자동으로 다시 컴파일할 수 있게 합니다.

```
tsc --watch
```

TypeScript 버전 4.9부터 파일 모니터링은 주로 파일 시스템 이벤트에 의존하며, 이벤트 기반 워처를 설정할 수 없는 경우 자동으로 폴링 방식으로 전환합니다.

Non-null 단언 연산자

확정 할당 단언이라고도 하는 non-null 단언 연산자(후위 !)는 TypeScript의 정적 타입 분석에서 변수나 프로퍼티가 null 또는 undefined일 수 있다고 판단하더라도 null이나 undefined가 아니라고 단언할 수 있게 하는 TypeScript 기능입니다. 이 기능을 사용하면 명시적인 검사를 모두 제거할 수 있습니다.

```
type Person = {
    name: string;
};

const printName = (person?: Person) => {
    console.log(`Name is ${person!.name}`);
};
```

기본값 선언

기본값 선언은 변수나 매개변수에 기본값을 할당할 때 사용됩니다. 즉, 해당 변수나 매개변수에 값이 제공되지 않으면 기본값이 대신 사용됩니다.

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

옵셔널 체이닝

옵셔널 체이닝 연산자 ?.는 프로퍼티나 메서드에 접근할 때 일반 점 연산자(.)처럼 작동합니다. 하지만 오류를 발생시키는 대신 표현식 평가를 종료하고 undefined를 반환하여 null 또는 undefined 값을 안전하게 처리합니다.

```ts
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};

console.log(person.address?.city); // undefined
```

널 병합 연산자

널 병합 연산자 ??는 왼쪽 값이 null 또는 undefined이면 오른쪽 값을 반환하고, 그렇지 않으면 왼쪽 값을 반환합니다.

```ts
const foo = null ?? 'foo';
console.log(foo); // foo

const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

템플릿 리터럴 타입

템플릿 리터럴 타입을 사용하면 타입 수준에서 문자열 값을 조작하고 기존 문자열 타입을 기반으로 새로운 문자열 타입을 생성할 수 있습니다. 문자열 기반 연산으로 더 표현력 있고 정확한 타입을 만드는 데 유용합니다.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type  Id  =  `${Department}-${Language}-id`;  //  "engineering-english-id"  |
"engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

함수 오버로딩

함수 오버로딩을 사용하면 동일한 함수 이름에 대해 각각 서로 다른 매개변수 타입과 반환 타입을 가진 여러 함수 시그니처를 정의할 수 있습니다. 오버로드된 함수를 호출하면 TypeScript는 제공된 인수를 사용하여 올바른 함수 시그니처를 결정합니다.

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
    throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

재귀 타입

재귀 타입은 자기 자신을 참조할 수 있는 타입입니다. 연결 리스트, 트리, 그래프처럼 계층적이거나 재귀적인 구조(잠재적으로 무한히 중첩될 수 있는 구조)를 가진 데이터 구조를 정의할 때 유용합니다.

```
type ListNode<T> = {
    data: T;
```

```
    next: ListNode<T> | undefined;
};
```

재귀 조건부 타입

TypeScript에서는 논리와 재귀를 사용해 복잡한 타입 관계를 정의할 수 있습니다. 간단히 나누어 살펴보겠습니다.

조건부 타입을 사용하면 불리언 조건에 따라 타입을 정의할 수 있습니다.

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number';
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

재귀란 타입 정의 안에서 자기 자신을 참조하는 타입 정의를 의미합니다.

```
type Json = string | number | boolean | null | Json[] | { [key: string]:
Json };

const data: Json = {
    prop1: true,
    prop2: 'prop2',
    prop3: {
        prop4: [],
    },
};
```

재귀 조건부 타입은 조건부 논리와 재귀를 결합합니다. 즉, 타입 정의가 조건부 논리를 통해 자기 자신에 의존할 수 있으므로 복잡하고 유연한 타입 관계를 만들 수 있습니다.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;
```

```
type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 | 1 | 6
```

Node의 ECMAScript 모듈 지원

Node.js는 버전 15.3.0부터 ECMAScript 모듈을 지원하며, TypeScript는 버전 4.7부터 Node.js용 ECMAScript 모듈을 지원합니다. tsconfig.json 파일에서 `module` 프로퍼티의 값을 `nodenext`로 설정하면 이 지원을 활성화할 수 있습니다. 예시는 다음과 같습니다.

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

Node.js는 모듈에 두 가지 파일 확장자를 지원합니다. ES 모듈에는 `.mjs`, CommonJS 모듈에는 `.cjs`를 사용합니다. TypeScript에서 이에 해당하는 파일 확장자는 ES 모듈의 경우 `.mts`, CommonJS 모듈의 경우 `.cts`입니다. TypeScript 컴파일러가 이러한 파일을 JavaScript로 트랜스파일하면 `.mjs` 및 `.cjs` 파일이 생성됩니다.

프로젝트에서 ES 모듈을 사용하려면 package.json 파일에서 `type` 프로퍼티를 "module"로 설정할 수 있습니다. 이렇게 하면 Node.js가 해당 프로젝트를 ES 모듈 프로젝트로 취급합니다.

또한 TypeScript는 .d.ts 파일의 타입 선언도 지원합니다. 이러한 선언 파일은 TypeScript로 작성된 라이브러리나 모듈에 대한 타입 정보를 제공하므로, 다른 개발자가 TypeScript의 타입 검사 및 자동 완성 기능과 함께 해당 라이브러리나 모듈을 사용할 수 있습니다.

단언 함수

TypeScript에서 단언 함수는 반환 값을 바탕으로 특정 조건이 검증되었음을 나타내는 함수입니다. 가장 간단한 형태의 단언 함수는 주어진 조건자를 검사하고, 조건자가 false로 평가되면 오류를 발생시킵니다.

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

또는 함수 표현식으로 선언할 수 있습니다.

```
type AssertIsNumber = (value: unknown) => asserts value is number;
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
};
```

단언 함수는 타입 가드와 유사합니다. 타입 가드는 런타임 검사를 수행하고 특정 범위 내에서 값의 타입을 보장하기 위해 처음 도입되었습니다. 구체적으로 타입 가드는 타입 조건자를 평가하고 그 조건자가 참인지 거짓인지를 나타내는 불리언 값을 반환하는 함수입니다. 조건자가 충족되지 않을 때 false를 반환하는 대신 오류를 발생시키려는 단언 함수와는 약간 다릅니다.

타입 가드의 예시는 다음과 같습니다.

```
const isNumber = (value: unknown): value is number => typeof value ===
'number';
```

가변 튜플 타입

가변 튜플 타입은 TypeScript 버전 4.0에서 도입된 기능입니다. 먼저 튜플이 무엇인지 다시 살펴보겠습니다.

튜플 타입은 길이가 정해져 있고 각 요소의 타입이 알려진 배열입니다.

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

"가변"이라는 용어는 정해지지 않은 인수 개수(가변 개수의 인수를 허용함)를 의미합니다.

가변 튜플은 앞에서 설명한 모든 특성을 가지지만, 정확한 형태가 아직 정의되지 않은 튜플 타입입니다.

```
type Bar<T extends unknown[]> = [boolean, ...T, number];

type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

앞의 코드에서 튜플의 형태가 전달된 제네릭 T에 의해 정의되는 것을 볼 수 있습니다.

가변 튜플은 여러 제네릭을 받을 수 있으므로 매우 유연합니다.

```
type Bar<T extends unknown[], G extends unknown[]> = [...T, boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean, boolean]
```

새로운 가변 튜플을 사용하면 다음이 가능합니다.

- 이제 튜플 타입 구문의 스프레드가 제네릭일 수 있으므로, 실제로 다루는 타입을 알지 못하더라도 튜플과 배열에 대한 고차 연산을 표

현할 수 있습니다.
- 나머지 요소는 튜플의 어느 위치에나 올 수 있습니다.

예시:

```
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
): [...T, ...U] {
    return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

박싱 타입

박싱 타입은 원시 타입을 객체로 나타내는 데 사용되는 래퍼 객체를 의미합니다. 이러한 래퍼 객체는 원시 값에서 직접 사용할 수 없는 추가 기능과 메서드를 제공합니다.

charAt이나 normalize 같은 메서드에 string 원시 값으로 접근하면 JavaScript는 해당 값을 String 객체로 감싸고 메서드를 호출한 다음 그 객체를 버립니다.

예시:

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

TypeScript는 원시 타입과 그에 해당하는 객체 래퍼에 별도의 타입을 제공하여 이러한 차이를 나타냅니다.

- string => String
- number => Number
- boolean => Boolean
- symbol => Symbol
- bigint => BigInt

일반적으로 박싱 타입은 필요하지 않습니다. 박싱 타입을 사용하지 말고 대신 원시 타입을 사용하세요. 예를 들어 `string`을 `String` 대신 사용합니다.

TypeScript의 공변성과 반공변성

공변성과 반공변성은 제네릭 타입 안에서 타입 관계가 어떻게 적용되는지 설명합니다. `Dog`는 `Animal`이지만 모든 `Animal`이 `Dog`는 아닙니다.

공변성(값 생성): `Dog`를 반환하는 함수는 `Animal`을 반환하는 함수가 필요한 곳에서 사용할 수 있습니다. 모든 개는 동물이지만 임의의 동물을 반환하는 함수는 고양이를 반환할 수도 있습니다.

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
```

```ts
declare let produceDog: Producer<Dog>;

produceAnimal = produceDog; // Valid
produceDog = produceAnimal; // Error
```

반공변성(값 소비): 모든 Animal을 받는 함수는 Dog를 받는 함수가 필요한 곳에서 사용할 수 있습니다. 개도 처리할 수 있지만 개만 받는 함수는 모든 동물을 안전하게 처리할 수 없습니다.

```ts
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;

consumeDog = consumeAnimal; // Valid
consumeAnimal = consumeDog; // Error with strictFunctionTypes
```

화살표는 안전한 할당 방향을 나타냅니다.

- Dog -> Animal
- Producer<Dog> -> Producer<Animal> (covariance)
- Consumer<Animal> -> Consumer<Dog> (contravariance)

변경 가능한 배열: TypeScript는 Dog[]를 Animal[]에 할당하는 것도 허용합니다. 두 변수는 같은 배열을 참조하므로 더 넓은 타입을 통해 개가 아닌 Animal을 추가할 수 있습니다.

```ts
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // Allowed, but unsafe
// dogs now contains an Animal that is not a Dog.
```

컴파일러 옵션: strictFunctionTypes를 사용하면 독립적인 함수의 매개변수를 반공변적으로 검사합니다. 사용하지 않으면 안전하지 않아도 양방향을 허용할 수 있습니다(양변성). 메서드와 생성자 선언의 매개변수는 예외입니다.

타입 매개변수의 선택적 변성 어노테이션

TypeScript 4.7부터 out은 공변성, in은 반공변성, in out은 불변성을 나타냅니다. 보통 TypeScript가 변성을 추론합니다. 주석은 제네릭 매개변수의 실제 사용 방식과 일치해야 하며 동작을 임의로 바꾸지는 않습니다.

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

템플릿 문자열 패턴 인덱스 시그니처

템플릿 문자열 패턴 인덱스 시그니처를 사용하면 템플릿 문자열 패턴으로 유연한 인덱스 시그니처를 정의할 수 있습니다. 이 기능을 통해 특정 문자열 키 패턴으로 인덱싱할 수 있는 객체를 생성할 수 있으므로, 프로퍼티에 접근하고 프로퍼티를 조작할 때 더 세밀하게 제어하고 구체적으로 지정할 수 있습니다.

TypeScript는 버전 4.4부터 심볼과 템플릿 문자열 패턴을 위한 인덱스 시그니처를 지원합니다.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
```

```
    [uniqueSymbol]: string;
    [key: MyKeys]: number;
};

const obj: MyObject = {
    [uniqueSymbol]: 'Unique symbol key',
    'key-a': 123,
    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Unique symbol key
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

satisfies 연산자

satisfies 연산자를 사용하면 주어진 타입이 특정 인터페이스나 조건을 충족하는지 확인할 수 있습니다. 다시 말해, 타입이 특정 인터페이스에 필요한 모든 프로퍼티와 메서드를 갖추도록 보장합니다. 이는 변수가 타입 정의에 부합하는지 보장하는 방법입니다. 예시는 다음과 같습니다.

```
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// Type Annotation using `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// In the following lines, TypeScript won't be able to infer properly
user.attributes?.map(console.log); // Property 'map' does not exist on type
'string | string[]'. Property 'map' does not exist on type 'string'.
```

```
user.nickName; // string | string[] | undefined

// Type assertion using `as`
const user2 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// Here too, TypeScript won't be able to infer properly
user2.attributes?.map(console.log); // Property 'map' does not exist on type
'string | string[]'. Property 'map' does not exist on type 'string'.
user2.nickName; // string | string[] | undefined

// Using the `satisfies` operator we can properly infer the types now
const user3 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log); // TypeScript infers correctly: string[]
user3.nickName; // TypeScript infers correctly: undefined
```

타입 전용 가져오기와 내보내기

타입 전용 가져오기와 내보내기를 사용하면 해당 타입과 관련된 값이나 함수를 가져오거나 내보내지 않고 타입만 가져오거나 내보낼 수 있습니다. 이는 번들 크기를 줄이는 데 유용할 수 있습니다.

타입 전용 가져오기를 사용하려면 `import type` 키워드를 사용하면 됩니다.

TypeScript에서는 `allowImportingTsExtensions` 설정과 관계없이 타입 전용 가져오기에서 선언 파일과 구현 파일의 확장자(.ts, .mts, .cts, .tsx)를

모두 사용할 수 있습니다.

예시:

```
import type { House } from './house.ts';
```

다음과 같은 형식이 지원됩니다.

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
export type { T };
export type { T } from './mod';
```

using 선언과 명시적 리소스 관리

using 선언은 const와 유사한 블록 범위의 불변 바인딩으로, 폐기 가능한 리소스를 관리하는 데 사용됩니다. 값으로 초기화되면 해당 값의 Symbol.dispose 메서드가 기록되고, 이후 둘러싸는 블록 범위를 벗어날 때 실행됩니다.

이는 객체 생성 후 연결 종료, 파일 삭제, 메모리 해제와 같은 필수 정리 작업을 수행하는 데 유용한 ECMAScript의 리소스 관리 기능을 기반으로 합니다.

참고:

- TypeScript 5.2에 최근 도입된 기능이므로 대부분의 런타임은 네이티브로 지원하지 않습니다. Symbol.dispose, Symbol.asyncDispose, DisposableStack, AsyncDisposableStack, SuppressedError에는 폴리필이 필요합니다.
- 또한 tsconfig.json을 다음과 같이 구성해야 합니다.

```
{
    "compilerOptions": {
        "target": "es2022",
        "lib": ["es2022", "esnext.disposable", "dom"]
    }
}
```

예시:

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Simple polyfill

const doWork = (): Disposable => {
    return {
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // Resource is declared
    console.log(2);
} // Resource is disposed (e.g., `work[Symbol.dispose]()` is evaluated)

console.log(3);
```

코드의 로그 출력은 다음과 같습니다.

```
1
2
disposed
3
```

폐기할 수 있는 리소스는 Disposable 인터페이스를 준수해야 합니다.

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

using 선언은 리소스 폐기 작업을 스택에 기록하여 선언의 역순으로 폐기되도록 합니다.

```
{
    using j = getA(),
        y = getB();
    using k = getC();
} // disposes `C`, then `B`, then `A`.
```

이후 코드가 실행되거나 예외가 발생하더라도 리소스는 반드시 폐기됩니다. 이 과정에서 폐기 작업이 예외를 던져 다른 예외를 억제할 수 있습니다. 억제된 오류에 대한 정보를 유지하기 위해 새로운 네이티브 예외인 SuppressedError가 도입되었습니다.

await using 선언

await using 선언은 비동기적으로 폐기 가능한 리소스를 처리합니다. 값에는 Symbol.asyncDispose 메서드가 있어야 하며, 블록이 끝날 때 이 메서드의 완료를 기다립니다.

```
async function doWorkAsync() {
    await using work = doWorkAsync(); // Resource is declared
} // Resource is disposed (e.g., `await work[Symbol.asyncDispose]()` is evaluated)
```

비동기적으로 폐기 가능한 리소스는 Disposable 또는 AsyncDisposable 인터페이스를 준수해야 합니다.

```ts
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose ??= Symbol('Symbol.asyncDispose'); // Simple polyfill

class DatabaseConnection implements AsyncDisposable {
    // A method that is called when the object is disposed asynchronously
    [Symbol.asyncDispose]() {
        // Close the connection and return a promise
        return this.close();
    }

    async close() {
        console.log('Closing the connection...');
        await new Promise(resolve => setTimeout(resolve, 1000));
        console.log('Connection closed.');
    }
}

async function doWork() {
     // Create a new connection and dispose it asynchronously when it goes out of scope
      await using connection = new DatabaseConnection(); //  Resource is declared
    console.log('Doing some work...');
} // Resource is disposed (e.g., `await connection[Symbol.asyncDispose]()` is evaluated)

doWork();
```

코드는 다음과 같은 로그를 출력합니다.

```
Doing some work...
Closing the connection...
```

```
Connection closed.
```

using 및 await using 선언은 for, for-in, for-of, for-await-of, switch 문에서 사용할 수 있습니다.

가져오기 속성

TypeScript 5.3의 가져오기 속성(가져오기에 대한 레이블)은 런타임에 모듈(JSON 등)을 처리하는 방법을 알려 줍니다. 이를 통해 가져오기 방식이 명확해져 보안이 향상되고, 더 안전한 리소스 로딩을 위해 콘텐츠 보안 정책(CSP)에 부합합니다. TypeScript는 가져오기 속성이 유효한지 확인하지만, 특정 모듈을 처리하기 위해 이를 해석하는 일은 런타임에 맡깁니다.

예시:

```
import config from './config.json' with { type: 'json' };
```

동적 가져오기의 경우:

```
const config = import('./config.json', { with: { type: 'json' } });
```

정규 표현식 구문 검사

TypeScript 5.5.4부터는 컴파일 시간에 정규 표현식 리터럴의 일반적인 오류(예: 잘못된 구문, 잘못된 역참조, 대상 JavaScript 버전에서 지원되지 않는 기능)를 검사합니다. 버그를 더 일찍 발견하는 데 도움이 되지만, new RegExp("…") 문자열은 검사하지 않습니다.

```
let r = /(a)\2/; // Error: This backreference refers to a group that does not exist.
```

import defer

`import defer`를 사용하면 모듈을 로드하되 모듈에서 실제로 무언가를 사용할 때까지 실행을 지연할 수 있습니다. 이렇게 하면 불필요한 작업과 부작용을 피하는 데 도움이 됩니다.

- 다음 형식에서만 동작합니다: `import defer * as name from "module"`
- 내보낸 항목에 접근할 때만 코드가 실행됩니다.